Learning to Evolve Specifications: A Methodology for Interactive Verification and Adaptation

Rafael V. Borges, City University London
Artur d'Avila Garcez, City University London
Luis C. Lamb, Federal University of Rio Grande do Sul
Bashar Nuseibeh, The Open University, UK and Lero, Ireland

Formal verification plays an important role in improving the quality and safety of products and processes. We propose a new methodology that includes a tool that makes use of machine learning to evolve models, generating new models from initial descriptions given counter-examples from a verification process, or from examples of system runs when a model is not available. The methodology uses standard model checking for verifying properties against an initial model description. A machine learning tool was integrated with a model checker in order to adapt the model when a property is violated. The main characteristic of our methodology is that, differently from alternative related approaches, it is robust in that it is capable of coping with errors in the specification process due to the use of neural networks in its core, so that performance degrades gracefully as indicated by experiments. We validated the framework in a pump system case study and a real-world power plant safety specification. The results indicate that the tool, which makes the neural network implementation transparent to the user, is capable of learning to adapt specifications from system properties, learning from system runs when a property is not available, and learning even when errors are present in the specification. The tool was fully integrated with an open source model checker and is available for use and evaluation of our results.

Categories and Subject Descriptors: D.2.8 [**Software Engineering**]: Metrics—*complexity measures, performance measures*; H.4 [**Information Systems Applications**]: Miscellaneous

General Terms: Theory

Additional Key Words and Phrases: Requirements evolution, adaptation, machine learning, model verification

ACM Reference Format:
Borges, R. V., d'Avila Garcez, A., Lamb, L. C., Nuseibeh, B. 2012. Learning to Evolve Requirements Specifications. ACM Trans. Embedd. Comput. Syst. V, N, Article A (January YYYY), 30 pages.
DOI = 10.1145/0000000.0000000 http://doi.acm.org/10.1145/0000000.0000000

1. INTRODUCTION

Formal methods of specification and model verification play an increasingly important role in improving the quality and safety of products and processes in software development [Beyer et al. 2007; Bobaru et al. 2008; Clarke et al. 2009; Clarke et al. 2003; Deshmukh et al. 2009; Peled et al. 2001]. However, the specification process still involves considerable human effort and can be error prone [Nuseibeh et al. 2000; Parnas 2010]. Errors may accumulate at different phases of the process, from the requirements specification through to design and actual implementation of the software models.

Model verification tackles this problem by allowing the desired properties of the models to be verified. Model checking has become a successful approach to formal verification, but its industrial

This work is supported by CNPq (The Brazilian National Research Council).
Author's addresses: Rafael V. Borges and Artur d'Avila Garcez, Department of Computer Science, City University London, London EC1V OHB, United Kingdom (Rafael.Borges.1@soi.city.ac.uk and aag@soi.city.ac.uk); Luis C. Lamb, Institute of Informatics, Federal University of Rio Grande do Sul, Porto Alegre RS, 91501-970, Brazil (luislamb@acm.org); Bashar Nuseibeh, Department of Computing, The Open University, UK and Lero, Ireland (B.Nuseibeh@open.ac.uk)

DOI 10.1145/0000000.0000000 http://doi.acm.org/10.1145/0000000.0000000

take up could be improved by (i) automated processes to adapt and evolve the models according to the information obtained from the verification process, (ii) tools that could allow systems to be verified even when an abstract description of a model is not available, and (iii) tools that could integrate verification and adaptation even in the presence of errors.

The use of model checking has proved an effective option in increasing the reliability of complex, autonomous and safety-critical systems, where software systems have to manage a large number of different scenarios [Beyer et al. 2007; Clarke et al. 2009]. As pointed out in a recent manifesto [Kreiker et al. 2011], which summarises the recent developments and challenges in the area, *"the expectations towards analysis and verification methods are astonishing in the light of the known undecidability or intractability of the problems they are expected to solve; [however,] there is a high potential for improvement from a synergetic integration of model-based design tools with analysis and verification tools"* [Kreiker et al. 2011].

Aware of the above theoretical limitations on scalability, in this paper we seek to achieve a useful integration of verification and adaptation. In particular, "model checking has a natural application where the goal is to arrive at a good model of the desired system through successive approximations" [Chechik et al. 2003]. Our goal, therefore, is to propose one such novel model and show, despite the theoretical limitations, that it is capable of producing useful approximations in a real application. In this paper, we have chosen the area of power systems fault diagnosis to conduct such an application.

When specification errors or inconsistencies are found, current model checking tools provide limited information about what should be rectified in the target application or system under development. Normally, counter-examples of the specific properties that have not been verified are provided [Beer et al. 2009; Chaki et al. 2004; Groce and Kroening 2005]. Such information is then used by the developers or designers to revise the system's model before a new verification process is triggered, in a cycle that will continue until a validated specification is obtained.

Different techniques have been proposed to automate the evolution of specifications, but not in the presence of errors in the specification, as proposed in this paper. For instance, [Alrajeh et al. 2009a; Alrajeh et al. 2009b] describes symbolic tools that use deductive and abductive reasoning in order to adapt and evolve temporal models to satisfy a set of properties (given as desirable and undesirable sequences of events or as goal models); see also [Damas et al. 2009]. As illustrated by Peled et al. [Peled et al. 2001], the use of verification tools requires an abstract description of the system to follow a modeling language often specific and restricted to a given tool or system. Although, in this paper, we use a general temporal modelling language, we seek to ameliorate this problem in three ways.

First, our tool works when an initial description is not present, generating a model given system runs, as done in [Peled et al. 2001] so that the user is not forced to specify the system from the beginning in a predefined language. Second, as pointed out by Jeff Kramer [personal communication, 2012], the combination of symbolic and numerical approaches, as proposed here, should provide a language richness required by the complexity and scope of the problem at hand, which includes the need for a combination of structured reasoning, as done in [Alrajeh et al. 2009a; Alrajeh et al. 2009b], with unstructured reasoning for error-prone processes and methods, as proposed. Third, our tool works with a widely used and open source model checker [Cimatti et al. 1999] even given erroneous system descriptions.

Our approach is complementary to the one presented in [Alrajeh et al. 2009a; Alrajeh et al. 2009b; Alrajeh et al. 2012]: in their work, they make use of Inductive Logic Programming (ILP) [Muggleton and Raedt 1994] to generate missing operational requirements which are used in order to satisfy goals. ILP assumes correctness of background knowledge, so that errors in the descriptions are not allowed. Our approach makes use of neural learning, which offers a more fine-grained form of learning based on gradient-descent computations on real numbers [Haykin 1999], and performs with great effectiveness, learning tasks in autonomous and evolving systems. In order to build such an effective tool, that takes advantage of neural learning and symbolic model specifications, we combine symbolic and numerical reasoning through the use of a "neural-symbolic" approach [d'Avila Garcez et al. 2002] (explained below). This allows our tool to handle errors with robustness (since

neural networks have been consistently shown effective at robust learning [Haykin 1999; d'Avila Garcez et al. 2002]).

The use of a verification technique capable of learning an abstract description from the observed behavior of a system can help automate the verification and adaptation tasks described above and allow a more general application of formal verification tools. In order to integrate the above ideas of adaptation, evolution and verification, we propose the use of a machine learning technique, known as a neural-symbolic approach [d'Avila Garcez et al. 2003; d'Avila Garcez et al. 2009; Valiant 2003]. Neural-symbolic integration combines automated reasoning from symbolic artificial intelligence and robust machine learning through the use of neural networks and knowledge extraction. Pure neural networks have been shown very effective at performing robust learning in many applications, in particular in adaptive and evolving systems [d'Avila Garcez et al. 2009; Haykin 1999; Rumelhart et al. 1986]. However, a full integration with a symbolic model checker will require a neural-symbolic approach, as proposed in what follows.

Building upon the principles of neural-symbolic integration, we propose a methodology that allows the neural adaptation, verification and evolution of models, and a full integration with a symbolic model checking tool. In what regards model evolution, this verification and adaptation (V&A) framework is capable of generating an improved model from an original description and the counter-examples produced by the model checker. The framework also works when an initial description is not available by learning an abstract description based only on examples of the observed behavior of the system. Through the use of a neural-symbolic methodology, the network implementation is transparent to the user, and interactions with the system takes place mostly with the use of state transition diagrams.

In neural-symbolic integration, translation algorithms are applied to convert symbolic knowledge (e.g. if-then rules or logical specifications) into connectionist networks (e.g. recurrent neural networks) and vice-versa. The translation from symbolic rules into networks provides a way of including explicit knowledge into the networks, which has been shown to improve network learning performance [d'Avila Garcez et al. 2002]. The translation from a trained network back into rules provides descriptions of the behaviour of the network and its learning process, offering explanations to the networks (a long-standing criticism of neural networks has been the lack of explanation capacity, which this process of rule extraction offers). In between translations, a neural-symbolic system functions exactly like a neural network, which can evolve and adapt to new situations by training and generalizing from examples. Neural-symbolic systems have been shown more effective and robust than purely-symbolic systems at learning from examples because of this use of a network-based training [d'Avila Garcez et al. 2002].

The main contribution of our methodology through the V&A framework is in unifying, in a robust and sound process, the verification of properties of a given model, the evolution of this model according to such properties and the possibility of building this knowledge model from the observation of an actual system. Our experimental results using a real power plant application as well as benchmark studies show that the tool is capable of learning to adapt specifications given system properties, learning from system runs when a property is not available, and robust learning in the presence of errors in the specification.

The paper is structured as follows. Section 2 presents background and related work. Section 3 describes the framework, focusing on a theory of adaptation and evolution of models and their integration with the model checker. Section 4 presents the validation of our framework in a number of case studies, including a real-world scenario application in a power plant. Section 5 concludes and points out directions for further research.

2. BACKGROUND AND RELATED WORK

In this section, we summarise the background material on temporal reasoning, verification and learning used throughout the paper, and compare and contrast the work presented here with the main related work in the area.

2.1. Temporal Models of Systems

The use of temporal models and logics have found a large number of applications in Computer Science since the pioneering work of Pnueli [Clarke et al. 2009; Fisher et al. 2005; Gabel and Su 2008; Lamb et al. 2007; Pnueli 1977]. Propositional logics, when extended with modal operators to represent a temporal dimension, provide a powerful language for the representation and inference of a broad range of dynamic and temporal models [Clarke et al. 2009; Lamb et al. 2007; Pnueli 1977]. Among these modal systems, LTL (linear temporal logics) and CTL (computational tree logics) [Ben-Ari et al. 1981] are broadly used in different domains. In these languages, a set of modal operators allow an expressive representation of temporal relations, considering both a linear view of the time flow (LTL) or an branching time approach (CTL), where each time point might have different possible futures. Model Checking is probably one of the most successful applications of temporal logics in Software Engineering. It consists of a set of automated tools to perform formal verification of computing systems [Beyer et al. 2007; Clarke et al. 2009]. In model checking, the system is described as a temporal model, in which the satisfiability of a property described in a temporal logic can be verified automatically.

Model checking has the advantage of formal static verification tools (when compared with the dynamic process of testing). It is expected to reduce the need for human intervention, such as in proof-based verification systems, despite the known undecidability or intractability of the problems they are expected to solve and the associated scalability issues [Clarke et al. 2009; Fisher et al. 2005]. Model checking tools usually have three components: A *description language* used to represent the model, a *specification language* to represent the properties that should be satisfied by the model and the *verification engine* that will perform the actual verification. An example of model checker is the New Symbolic Model Verification (NuSMV) [Cimatti et al. 1999; Cimatti et al. 2002], which gathers several different functionalities in an unique tool. Besides having an expressive description language, NuSMV allows the verification of properties expressed in both CTL and LTL, through different techniques based on binary decision diagrams (BDD) [Yang et al. 1998] or on propositional satisfiability verification (SAT) [Cimatti et al. 2002].

2.2. Integrating Verification and Adaptation

Machine learning, the automated acquisition of knowledge, is an important development over the last decades in Computer Science, with several applications in different areas of Software Engineering [Cleland-Huang et al. 2010; Dumitru et al. ; Zhang and Tsai 2005]. Different cases of symbolic verification techniques extended with learning, adaptation or evolution strategies can be found in the literature. For instance, [Peled et al. 2001] describes black box checking, a process to perform formal verification when abstract models of the system are not available. The process consists in building a high level description of a system through the observation of its behaviour, in such a way that the learned model can be used for purposes such as automatic verification. An iterative approach to model checking is CEGAR - Counter-example Guided Abstraction Refinement [Clarke et al. 2003]. In this approach, the counter-examples generated in the process of model checking are used to refine abstractions. The use of abstractions is considered an important resource in order to tackle the state-explosion problem in formal verification tools. The evolution of a system's description according to properties to be satisfied, also, has been tackled by different symbolic approaches: [d'Avila Garcez et al. 2001] propose a cycle of analysis and revision in order to evolve requirements specifications. Also, abductive and deductive systems were used by [Alrajeh et al. 2009a; Alrajeh et al. 2009b] to improve models described in event calculus according to positive or negative examples of desired behaviour, or abstract description of properties such as goal models; [Alrajeh et al. 2009a; Alrajeh et al. 2009b] have shown that the application of such techniques may improve different processes in the software cycle, such as the evolution of requirement descriptions according to examples given by stakeholders. Alrajeh et al [Alrajeh et al. 2012] also propose a methodology for the identification of incompleteness in operational requirements. Alrajeh et al do so by using model checking and inductive logic programming (ILP). In their work, model checking is used to

produce counterexamples when incompleteness are detected in (operational) requirements specifications. Our approach can also deal with errors (observed behaviours that do not correspond to the actual desired behaviour of the system, or problems detected when recording observations of the system) in the specification, which are not addressed in their work. As regards learning, [Alrajeh et al. 2012] use ILP to generate missing operational requirements that may be needed to satisfy goals. In our work, the learning engine is used to identify not only mistakes in the specifications, but also to generate models (that can be refined) from system runs. Therefore, we see the approaches as complementary: in addition to identifying errors in the specifications, we can also generate initial models from sets of systems runs. We provide a quantitative analysis of the accuracy and confidence of our model in Section 4.

While several symbolic tools have been proposed to allow learning and adaptation in a number of application areas, a main criticism in the literature has been that symbolic systems would be too brittle to accomplish high performance on large-scale, error-prone data, including robustness under uncertainty [Hitzler et al. 2004; Michalski 1987; Valiant 2008]. An alternative approach consists of neural networks: computational models inspired by the brain, where the adaptation task is performed through consecutive/incremental slight adaptations of numerical parameters according to the presentation of examples [d'Avila Garcez et al. 2009; Rumelhart et al. 1986]. The subtlety of the changes in the evolution of the knowledge, as well as the distribution of the processing in a parallel architecture, cater for greater robustness in the process of learning, with good performance even when errors are included or information is missing altogether [Haykin 1999]. Another advantage of neural networks is the possibility of performing complex computations: recurrent neural networks are computationally equivalent to Turing Machines [Lin et al. 1996], while being highly efficient; the speed to compute an output depends only on the structure of the network, with no increase in complexity when the system acquires new information [Rumelhart et al. 1986].

However, the advantages of connectionist systems may come with a drawback, that consists in the representation of the knowledge in the networks. The learned knowledge is coded in the form of numerical weights in the structure, and therefore cannot be fully understood by an external entity [Andrews et al. 1995]. In order to extend the applicability of neural networks to symbolic systems and models (such as the verification and evolution of software models), the neural-symbolic approach has been broadly considered as an important development in the definition of robust learning and reasoning systems [Valiant 2008]. A relevant strategy of neural-symbolic integration consists in translating the knowledge between symbolic and connectionist representations. Several authors have proposed different algorithms to represent symbolic knowledge in the form of neural networks [d'Avila Garcez et al. 2002; Lamb et al. 2007] and also to extract knowledge from a trained network into a set of logical rules [Andrews et al. 1995]. We will use the neural-symbolic methodology to overcome some of the limitations of purely symbolic learners in the process of model adaptation.

The main idea of the V&A framework was introduced in overview in [Borges et al. 2010a], and further developed in [Borges et al. 2011a] to include initial experimental results, translation algorithms between NuSMV and temporal logic rules, and extraction diagrams. The neural-symbolic translation algorithms for temporal rules used by the V&A framework were first introduced in [Borges et al. 2010b] and further developed in [Borges et al. 2011b] for the case of deterministic systems. Here, we also consider non-determinism. This paper is the first to present our methodology in comprehensive fashion and the entire framework in a self-contained way, including all the necessary translations from NuSMV to temporal rules, networks and extraction diagrams, discussing all the implementation details, including learning of non-deterministic knowledge, translation of scalar variables, the tool support and its validation, and providing a full evaluation of the framework in three testbeds, including a real-world power plant safety specification, black box checking and property-based adaptation.

3. V&A: A FRAMEWORK FOR VERIFICATION AND ADAPTATION

In this section, we introduce the Verification and Adaptation framework, describing in detail its learning and representation algorithms, and how knowledge extraction is used to reconstruct the learned models.

3.1. General Framework

Our methodology includes a framework that is able to integrate verification and adaptation of software descriptions. The verification is performed by a model checker [Cimatti et al. 2002]. The adaptation is tackled by a neural network, which is extended by symbolic modules to allow the integration of different forms of knowledge representations [d'Avila Garcez et al. 2009]. The framework is illustrated in Fig. 1. At its core is the learning engine, which consists of a neural-symbolic system that uses a variation of the NARX recurrent neural network model [Siegelmann et al. 1997] and is capable of incorporating three sources of knowledge for the representation, learning and evolution of temporal models, as detailed below.

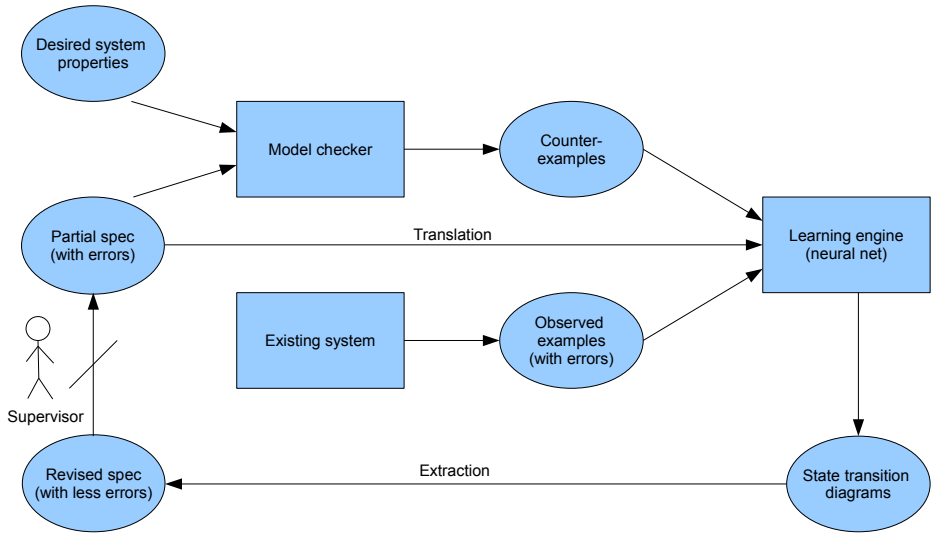

Fig. 1. General diagram of the proposed framework

The first source of knowledge consists of an initial description of a system, which might be incomplete or incorrect (see Fig. 1). This description contains explicit definitions of acceptable runs, and is called a specification (or *spec* for short, as in the figure). This initial model is expressed using the model checker description language, and translated into a temporal logic program to be used as background knowledge by the learning engine. The learning process is performed by integrating background knowledge with two other sources of knowledge, obtained either from a model checker or from existing system runs, and provided as a set of examples of a desired behaviour of the system (an example is an input-output pattern to be used by our neural-network training, denoting one transition in a state diagram; an example describes, therefore, that given a situation and an action at time t, a given situation is expected or desired at time $t + 1$). Observed examples obtained from existing system runs may also contain errors, but examples derived from the counter-examples produced by the model checker are assumed to respect the desired system properties, which are assumed to be correct (a counter-example describes that given a situation and an action at time t, a given situation

should not occur at time $t + 1$; we will describe how the framework uses counter-examples and system properties in more detail below). The learning process may, therefore, revise the spec in the light of the examples, which take priority. The framework also works when provided with sequences of examples, which we shall call *events*, which specify partial sequences of transitions that the model should satisfy. These sequences of arbitrary size n are partial because they may describe underspecified situations (i.e. containing unknown values) over a number of time points from t to $t + n - 1$. The network is, therefore, expected to combine and generalize such underspecified situations into a learned spec that, eventually, will satisfy the desired system properties, since the examples take precedence over the original spec and should satisfy the properties themselves. Although a proof of convergence does not exist, our experiments have shown that this iterative process can be useful in practice, leading to specs that satisfy the required properties. At such point the framework would not produce any counter-example as a result of its verification step. Whilst the verification of a system formally proves that it conforms with its specification, the verification step in the V&A framework proves that the system spec conforms with its desired properties, i.e. that each property p holds in the spec, written $spec \vdash p$. Notice that if background knowledge about the system is not available, the learning process can start by setting up a network randomly and training it with examples of system runs only (see Fig. 1). After the learning process, the learning engine should represent the knowledge learned which can be visualized by the user through a list of state transitions, which are associated with numerical values providing a measure of their importance (this visualization and extraction process will be detailed later).

The framework does not require the presence of all the knowledge sources, therefore accommodating different adaptation tasks in the domain of Software Engineering. The process of learning from examples allows the system to build a knowledge model (our *revised spec*) to represent an observed system. By translating the learned transitions into a NuSMV description, the model checker can be used in the verification of this observed system, performing a so-called black box checking [Peled et al. 2001]. On the other hand, if a knowledge description (our *partial spec*) is provided, the model checker can verify if it satisfies the desired properties, returning a counter-example when it does not. The framework can adapt such counter-examples to evolve the partial spec, learning a new revised model (*revised spec*) from examples. From this cycle of verification and adaptation, the properties are eventually expected to be satisfied by a revised spec since the derivation of the examples is guided by the properties (which are the only thing assumed to be correct and error-free in the model). The learning process seeks to generalise the examples into a useful, property-satisfying specification. This allows the process of model evolution guided by properties [Alrajeh et al. 2009b]. The tool support for the framework is available at http://vega.soi.city.ac.uk/~abct616/?cont=2, and includes a complete tool implementation with examples and results of the experiments reported in this paper. In what follows, the different possibilities will be discussed in turn.

3.2. Translation, Learning and Extraction (overview)

The learning core of the V&A framework uses the Sequential Connectionist Temporal Logic system [Lamb et al. 2007]. SCTL allows the use of a symbolic logic representation to describe the models in a neural network. It is capable of learning in scenarios where incomplete or incorrect information is presented. In the next paragraphs, we give a general description of the system focusing on what had to be changed to integrate it with a model checker and, hence, implement the V&A framework.

Definition 3.1. A **SCTL temporal model** $\mathcal{P}$ is a tuple $\mathcal{P} = \left\{ St^{\mathcal{P}}, In^{\mathcal{P}}, Init^{\mathcal{P}}, C^{\mathcal{P}} \right\}$, where $St^{\mathcal{P}}$ is the set of state variables α, $In^{\mathcal{P}}$ is the set of input variables β, $Init^{\mathcal{P}}$ is the initial state, defined by a mapping from $In^{\mathcal{P}} \cup St^{\mathcal{P}}$ to $\{true, false\}$ and $C^{\mathcal{P}}$ is a set of logical clauses of the form $\bigcirc\alpha \leftarrow \alpha_1, ..., \alpha_n, \beta_1, ..., \beta_m$. The symbol $\bigcirc$ is used here to denote the next time point; α will be true at the next time point if $\alpha_1, ..., \alpha_n, \beta_1, ..., \beta_m$ are all true at the current time point.

Any temporal model described in SCTL can be used to generate background knowledge for a learning engine composed of a recurrent neural network. By setting the numeric parameters

(weights) of the network to specific values according to the knowledge represented by the temporal model, the model can be *encoded* in the network, as detailed below. Networks can also be generated using random weights, i.e. without background knowledge. This will be useful when we intend to learn the description of a system by observing its behaviour only.

Each training example for the neural network is defined as the assignment of numerical values in $\{-1, 1\}$ to the input variables (-1 for false and 1 for true), and $\{-1, 0, 1\}$ to the state variables (-1 for false, zero denoting unknown, and 1 for true). More specifically, it is possible to have states that are not observable, and therefore no information about that state will be available to be used for learning. A sequence of examples can also be used for the purpose of learning. At each time point t, the numerical values for each example are assigned to the input variables, while the values of the state variables are defined according to the values obtained in the engine's output at the previous time point $t-1$. The network will define a set of values for state variables at the next time point $t+1$. These values are then compared to target values from the set of examples, and the difference (error signal) between the two will be used for the adaptation of the numerical parameters (weights) of the network. The change of weights in the network is essentially the learning process which seeks to reduce the error signal by progressively changing the function computed by the network. We use a variation of the widely used, standard gradient descent algorithm known as *backpropagation* through time to adapt the network's weights [Rumelhart et al. 1986]. It order to model the case where some state variables cannot be observed, the set of examples is allowed to contain values for subsets of the state variables with certain values missing. In this case, the error will be considered null for such variables with missing values, so that the adaptation process will occur as a function of the remaining variables.

Figure 2 shows a network obtained from four logical clauses C_1 to C_4. A logical-AND is computed from the input to the hidden layer of the network, and a logical-OR is computed from the hidden layer to the output, due to the weights W set in the network. Negative weights $-W$ are used to implement default negation ($\sim$) in the logical clauses. Input neurons D, E, F, G denote input variables. Input neurons B and C denote state variables (to which recurrent connections exist). Input vectors in $\{-1, 0, 1\}$ are presented to the network at each time point t. Output values are calculated and fed back to the network's state variables for a predefined number of time points or until a stable state is obtained (i.e. when the input and output values of each variable are the same; the network's outputs o_j are real numbers in the interval [-1,1] so that a threshold parameter $0 < A_{min} < 1$ is used to denote that an output is *true* if $A_{min} < o_j < 1$, $false$ if $-1 < o_j < -A_{min}$, and *unknown* otherwise. The network can be seen as computing traces in a state transition diagram given an initial state in $\{-1, 0, 1\}$, and at the same time computing the truth-values of output variable A for a number of time points.

For example, given $D = 1$, $E = -1$, $F = 1$ at time t, the network computes $B = 1$, $C = 1$ in parallel (at time $t + 1$). These values are fed back to the input and, given now $B = 1$, $C = 1$, the network computes $A = 1$ at time $t + 2$, which denotes that A is *true* now.

The above-mentioned learning process, whereby the network's parameters are adjusted according to the error signal in the network's outputs, is the method often used for adaptation in neural networks, and it has been shown to perform well in many different applications [Haykin 1999]. Besides this standard method, SCTL has another learning functionality, whereby learning is guided by the presentation of sequences or traces. This process is defined formally as follows.

Definition 3.2. A **learning sequence** $\mathcal{X}$ is a tuple $\mathcal{X} = \left\{S_0^{\mathcal{X}}, I^{\mathcal{X}}, S_n^{\mathcal{X}}\right\}$, where $S_0^{\mathcal{X}}$ is the initial state condition, $S_n^{\mathcal{X}}$ is the final state condition, and $I^{\mathcal{X}}$ consists of a sequence of input conditions $(I_0^{\mathcal{X}}, ..., I_{n-1}^{\mathcal{X}})$. Each condition assigns a *true* or $false$ value to a subset of the variables.

A state condition $S^{\mathcal{X}}$ is said to be satisfied by an assignment S^* of values to state variables if $S^*(\alpha) = S^{\mathcal{X}}(\alpha)$ for every α defined in $S^{\mathcal{X}}$, independently of the other state variables. In order to learn from sequences, at every time point t, the current state is compared with the initial state condition of all the given sequences. Every sequence $\mathcal{X}$ for which the initial condition $S_0^{\mathcal{X}}$ is satisfied

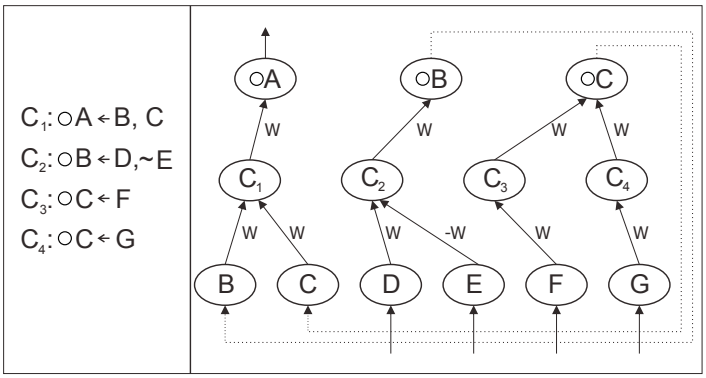

Fig. 2. A SCTL recurrent network

at t is inserted into a list of active properties. Sequences whose conditions are not satisfied are removed from the list, and sequences that were present throughout the entire input sequence I^X are used to define the desired output value, which will be used to determine the error signal for the backpropagation learning process. The framework allows both exclusive learning from sequences and the integrated learning of sequences and examples, providing mechanisms to deal with the absence of information and conflicts between different sources of information, as detailed below.

After the execution of the learning process, a new model description is stored in the network in the form of the numerical weights learned in a distributed way through the network structure. After learning, the network's weights are not restricted to values W and $-W$, as originally set, but could be any real number, as changed by backpropagation in order to minimise the network's errors. This distributed representation is responsible for the network's robustness, but at the same time it has been pointed out by several authors (e.g. [Andrews et al. 1995; Hitzler et al. 2004]) as a disadvantage, since the learned knowledge cannot be explained explicitly or used in the solution of analogous problems. SCTL contains a knowledge extraction module, which tackles this problem. The extraction module queries the trained network following an adequate probability distribution on the set of input sequences, to produce a state transition diagram, as defined below, which can be analyzed directly or translated back into a revised set of clauses like C_1 to C_4 above, as originally presented to the system.

Definition 3.3. A **transition network** $\mathcal{T}$ is a set of tuples $\left\{S_0^{\mathcal{T}}, I^{\mathcal{T}}, S_f^{\mathcal{T}}, w^{\mathcal{T}}, count^{\mathcal{T}}\right\}$, indicating transitions from a **source** $S_0^{\mathcal{T}}$ to a **target** $S_f^{\mathcal{T}}$ given an **input** $I^{\mathcal{T}}$. The values $w^{\mathcal{T}}$ and $count^{\mathcal{T}}$ are auxiliary extraction information, as defined below, representing, respectively, the weight and number of occurrences of each transition in the network.

The above transitions are obtained through the repeated presentation of a set of inputs to the network, and the calculation of the network's outputs. The process of selecting the inputs is the same as that used for learning: either a predefined set of existing examples is given to the network, or a method to automatically generate a sequence of inputs can be used. The values of $count^{\mathcal{T}}$ and *weight* ($w^{\mathcal{T}}$) are used as additional information to show which transitions are most relevant (more details about the extraction process will be provided later, including how one can further translate the obtained transition diagram into a set of temporal clauses).

3.3. Translation and Learning (details)

As mentioned above, in order to ground the framework on applications, we have chosen to use the NuSMV model checker. We also chose NuSMV because it is open source, so it could be integrated

Table I. The Simplified NuSMV Language

NuSMV Program ::
“MODULE” ModuleName “
“IVAR” VarDeclaration “;”
“VAR” VarDeclaration “;”
“ASSIGN”
InitBlock “;”
NextBlock “;”
VarDeclaration ::
VarId “:” VarType
— VarDeclaration “;” VarDeclaration
InitBlock ::
“init(” VarId “)=” ConstValue
— InitBlock “;” InitBlock
NextBlock
“Next(” VarId “)=” SimpleExp
— NextBlock “;” NextBlock
SimpleExpr ::
atom
— VarId
— BoolExpr
— CaseExpr
CaseExpr ::
“case” CondExpr “;” “esac”
CondExpr ::
BoolExpr “:” SimpleExpr
— CondExpr “;” CondExpr

into our tool, and widely used. For simplicity, first we consider the representation of deterministic models. We relax this restriction in the next section, thus increasing the applicability of the framework to nondeterministic scenarios. However, the purpose of the framework can already be verified in a deterministic scenario, as shown also in the next section.

Below, we formalise the simplified NuSMV language used, show how a specification described in this NuSMV language can be translated into temporal clauses and, hence, into SCTL networks. We allow two data types in the NuSMV language: boolean and scalar (i.e. a variable whose domain is defined by a finite enumeration). In practice, a model checker system performs a variable grounding before it does verification, converting every variable into a boolean. This propositional version of the model would be simpler to translate into SCTL, but the use of scalar variables offers a better generalization, as indicated by our experiments, through the use of a smaller network to represent and learn the model.

On the other hand, different constructs need to be incorporated into the SCTL system to integrate it with a model checker. The first of these consists of characterizing *groups* of variables so that only one variable from each group can be *true* at each time point. Groups of variables are useful when representing and learning the scalar variables in the outputs of the SCTL network. In the input of the SCTL network, each NuSMV scalar variable is replaced by a group of boolean variables, each one representing a possible value of the enumerated list (as done by NuSMV in practice). Table I specifies the simplified NuSMV language.

In order to translate *NextBlock* into SCTL, we need to make sure that scalar variables defined in the NuSMV model fit the structure of SCTL networks. All the comparisons regarding scalar variables (found in the body of the case expressions of the NuSMV language in Table I) can be directly treated as a boolean variable in the SCTL network, by considering expressions like $VarId = ConstValue$ as variables, and converting $VarId! = ConstValue$ into $\sim (VarId = ConstValue)$. When comparing different variables, all the possible instances of the variables need to be considered, thus creating more complex, but standard boolean expressions and networks. However, when dealing with state variables, this instantiation can be more complicated due to the assignment of values from a variable to another. In this case, to describe an expression like $VarId_1 := VarId_2$ in the

network, one needs to include a conditional statement, with a new condition to represent each possible value. The NuSMV model checker already performs this kind of conversion before verifying the model [Cimatti et al. 1999], and here we have used the same NuSMV mechanism. The Algorithm in Fig. 1 describes how the NuSMV headers can be used to generate those SCTL variables. Notice that the declaration of the variables remains the same before and after learning.

ALGORITHM 1: Processing Variable Declarations in NuSMV

Representation and Learning:

```
switch Declaration do
    case "IVAR" VarDeclaration ";"
        foreach VarId" :" VarType in VarDeclaration do
            if VarType = "boolean" then
                In^P ← In^P ∪ {VarId}
            else
                foreach v ∈ possibleValue(VarType) do
                    In^P ← In^P ∪ {VarId = v}
                end
        end
    case "VAR" VarDeclaration ";"
        foreach VarId" :" VarType in VarDeclaration do
            if VarType = "boolean" then
                St^P ← St^P ∪ {VarId}
            else
                foreach v ∈ possibleValue(VarType) do
                    St^P ← St^P ∪ {VarId = v}
                end
        end
    case "init(" VarId "):=" ConstValue
        if VarId is boolean then
            if ConstValue = "TRUE" then
                Init^P ← Init^P ∪ {VarId}
            else if ConstValue = "TRUE" then
                Init^P ← Init^P ∪ {~ VarId}
        else
            Init^P ← Init^P ∪ {VarId = ConstValue};
            foreach u ∈ possibleValue(x) do
                if VarId ≠ k then
                    Init^P ← Init^P ∪ {~ (VarId = ConstValue)}
            end
endsw
end
```

Once the above treatment of scalar comparisons is complete (see Fig. 1), the clauses of the SCTL network can be created. Our strategy consists of grouping all the conditions that lead to a boolean variable *VarId* being mapped to *true* into a single expression ϕ. Then, ϕ is rewritten as a ϕ' expression in disjunctive normal form (DNF) ($\phi' = \alpha_1 \vee \alpha_2 \vee ... \vee \alpha_n$, where each α_i is an expression

$\beta_i 1 \wedge \beta_i 2 \wedge ... \wedge \beta_i m$). Then, a clause is created for each conjunction α_i in ϕ': this clause will be of the form $\bigcirc(var = value) \leftarrow \alpha_1, \alpha_2, ..., \alpha_n$, denoting that variable *var* will have a certain value at time point $t + 1$ if $\alpha_1, \alpha_2, ..., \alpha_n$ all hold *true* at time point t.

Another important difference between NuSMV and SCTL regards the interpretation of a *list* of statements. In NuSMV, a conditional statement ($CaseBlock$) assumes that the conditions are read in a sequence, and when a condition is satisfied, the following ones are not read. In a neural-symbolic framework, knowledge is processed in parallel, meaning that two conditions (bodies of clauses) can be satisfied at the same time. Parallelism is a main attribute of a neural network, and one of the main reasons for its efficiency and robustness. Hence, we assume here that any such parallel conditions in the same $CaseBlock$ of a NuSMV model are mutually exclusive. An alternative to this assumption would be to include constraints explicitly as part of the derived clauses.

Our assumption of mutual exclusion above needs to be considered further in the case when no conditions of a $CaseBlock$ is true. Instead of implementing an "$Else$" definition, NuSMV uses a "TRUE" or "1" condition at the end of a block to indicate that if all the conditions before are $false$, the assignment given by this condition should be *true* instead. In SCTL, we do not use this interpretation, and if all the conditions in a $CaseBlock$ are $false$, no new value will be assigned to the related variable (i.e. the variable will keep its previous value). We claim that this approach is more appropriate in a temporal model of learning. SCTL works with "default negation", i.e. a variable is interpreted as $false$ unless there is a clause assigning it to *true*. Thus, we group all the conditions that lead a variable $VarId$ to $false$ in a boolean expression ψ. To represent the NuSMV cases when a previous variable value should be kept, we expand the ϕ expression defined above to $\phi \vee (VarId \wedge \neg\psi)$, where $\neg$ is used to denote (classical) negation, before converting it to DNF. This means that clauses will be inserted to deal with all conditions leading $VarId$ to *true*, and also the situation where $VarId$ was previously *true* and there are no conditions leading $VarId$ to $false$. The algorithm that translates NuSMV into SCTL networks is shown in Fig. 2.

ALGORITHM 2: Translation from NuSMV into SCTL networks

GenerateClauses:
- *Propositionalization*;
- **foreach** *declaration VarId " := " exp in NextBlock* **do**
  - **if** *exp is caseExp* **then**
    - **foreach** *declaration BoolExpr* : *v* **do**
      - **if** *v = "TRUE"* **then**
        - $\phi \leftarrow \phi \vee BoolExpr$
      - **else if** *v = "FALSE"* **then**
        - $\psi \leftarrow \psi \vee BoolExpr$
      - **else**
        - $\phi \leftarrow \phi \vee (BoolExpr \wedge v)$
    - **end**
  - **else if** *exp is VarId* **then**
    - $\phi \leftarrow exp$
  - $\phi' \leftarrow DNF(\phi \vee (VarId \wedge \sim psi))$
  - **foreach** $\alpha_i = (\beta_1 \wedge ... \wedge \beta_m)$ *in* $\phi' = (\alpha_1 \vee ... \vee \alpha_n)$ **do**
    - $Cl^P \leftarrow Cl^P \cup \{\bigcirc VarId \leftarrow beta_1, ..., beta_n\}$
  - **end**
- **end**

end

As mentioned above, the learning engine can learn from either general sequences or specific observations of the behavior of an existing system. Information can be provided simultaneously or separately to the SCTL network, allowing a range of applications. Below, we provide an example to

illustrate how this learning process is carried out. Consider a monitor of a resource that is supposed to allocate such a resource to different processes. Each process communicates with the monitor through a signal to request the resource (*Req*) and a signal to release it (*Rel*). These signals are input variables to the monitor, which also has a state variable for each process, to denote that the resource is allocated to the process. The following temporal model describes how such a monitor would work for one process A (the symbol $\bullet$ is used below to denote the previous time point, and is the dual of the $\circ$ (next time) symbol in temporal logic):

$$In^{\mathcal{P}} = \{Req_A, Rel_A\}$$
$$St^{\mathcal{P}} = \{A\}$$
$$Cl^{\mathcal{P}} = \{A \leftarrow Req_A;\ A \leftarrow \bullet A, \sim Rel_A\}$$

The above rules state that (i) if A requests the resource then it is allocated to A, (ii) if A got hold of the resource and did not release it then it contines to have it. Now, suppose we would like to extend this specification to handle two processes. Suppose, further, that instead of having to add rules by hand, and edit existing rules, we would like to use the V&A framework to learn those rules from observation of an existing monitor. In this case, we would extend the language to include the variables that are necessary to deal with the second process B, i.e. $In^{\mathcal{P}} = \{Req_A, Rel_A, Req_B, Rel_B\}$ and $St_{\mathcal{P}} = \{A, B\}$. We then would train a network, which represents our monitor, with examples of the resource allocation to process B; the training examples in Table II illustrate an allocation of the resource to B assuming no information in available in the examples about process A.

Table II. Examples of resource allocation to process B

Timepoint	Req_A	Rel_A	Req_B	Rel_B	A	B
1	1	-1	-1	-1	0	-1
2	-1	1	-1	-1	0	-1
3	-1	-1	1	-1	0	1
4	1	-1	-1	-1	0	1
5	-1	-1	-1	1	0	-1

The network would have 6 inputs $\{Req_A, Rel_A, Req_B, Rel_B, A, B\}$ and 2 outputs $\{A, B\}$. It is the job of backpropagation to adjust the network's weights so that whenever the input values of $\{Req_A, Rel_A, Req_B, Rel_B\}$ as in Table II are provided as input to the network, the target output values of $\{A, B\}$ in Table II are produced by the network's outputs. Output neurons $\{A, B\}$ are recurrently connected to input neurons $\{A, B\}$ and, hence, the network should also learn to generalise over time that, e.g., once $B = 1$, this value should persist until $Rel_B = 1$. SCTL networks can encode the temporal model explictly or learn a partial specification from examples; when provided with both, it would adjust the model to incorporate the examples, thus having to deal with the interaction between the two. For example, one would expect that A cannot be allocated the resource until it is released from B. This kind of property can be checked by the *verification* part of the framework, with any counter-example produced by the model checker being potentially useful in guiding the adaptation process further as part of a cycle which will be exemplified in detail in the next section.

In order to perform verification of the learned knowledge, allowing a full integration between SCTL and NuSMV, the trained network needs to be expressed back again in the form of a NuSMV model. This new model can be obtained by (i) extending the original declaration of input and state variables with transitions learned by the network, (ii) performing the actual task of rule extraction and (iii) expressing those rules in the form of the *NextBlock* of a NuSMV program. In what follows, we detail this process, which closes the cycle of learning and verification.

3.4. Reconstructing the Learned Model (extraction)

Given a network trained with some examples, the knowledge extraction process starts by querying the network to generate rules, and filtering the set of transitions to be considered according to their

current *count* and *weight* parameter values. The network might have included background knowledge in the form of rules which were revised by the set of examples. Returning to our *monitor* example, if a network containing the rules regarding process A above was trained with the examples in Table II regarding process B, one would expect to extract revised rules now describing the interaction between A and B, e.g.: $A \leftarrow Req_A, \sim B$.

Transitions occurring a small number of times may represent information that is too specific, and considering them can significantly increase the size of the extracted description, reducing its readability. Therefore, the choice of a threshold to filter the transitions according to their *count* offers a balance between comprehensibility and precision of the extracted models. On the other hand, filtering the transitions according to their *weight* allows the extracted model to disregard the situations where the learning process did not present a response with sufficient confidence to be included in the model. The concept of *confidence* is directly related to the real-numbered values in the network's outputs. Recall that the network's outputs produce values in the interval $\{-1, 1\}$; values that are close to either 1 or -1 are high-confidence values, as formally defined below.

To obtain the *NextBlock* for each state variable, we proceed as follows. When considering a variable α, let us call $K^+(\alpha)$ the group of all the transitions $\mathcal{T} = \left\{S_0^{\mathcal{T}}, I^{\mathcal{T}}, S_f^{\mathcal{T}}, w^{\mathcal{T}}, count^{\mathcal{T}}\right\}$, where α appears positively in $S_f^{\mathcal{T}}$, and $K^-(\alpha)$ the group of all the transitions where α appears negatively in $S_f^{\mathcal{T}}$ (recall Definition 3.3).

For each transition $\mathcal{T}$ as queried in the network, one can define a logical expression $pre(\mathcal{T})$ representing information about the current state and the input associated with that transition. The expression in $pre(\mathcal{T})$ will be the conjunction of all the elements in $S_0^{\mathcal{T}}$ and $I^{\mathcal{T}}$. For each group K, one can define a logical expression $DNF(K)$ in disjunctive normal form as the disjunction of $pre(\mathcal{T})$ for every $\mathcal{T} \in K$. Finally, $DNF(K)$ can be simplified with the use, e.g., of Karnaugh maps, and used to generate the NuSMV description about the next-time value of each α variable. The algorithm in Fig. 3 implements this process.

ALGORITHM 3: Translating SCTL Networks into NuSMV Models

Reconstruct Knowledge
  Filter relevant transitions $\mathcal{T}$.
  foreach $\alpha \in St^{\mathcal{P}}$ **do**
    foreach $\mathcal{T} = \left\{S_0^{\mathcal{T}}, I^{\mathcal{T}}, S_f^{\mathcal{T}}, w^{\mathcal{T}}, count^{\mathcal{T}}\right\}$ **do**
      if $S^{\mathcal{T}}(\alpha) = true$ **then** $K^+(\alpha) \leftarrow K(\alpha) \cup \{\mathcal{T}\}$;
      if $S^{\mathcal{T}}(\alpha) = false$ **then** $K^-(\alpha) \leftarrow K(\alpha) \cup \{\mathcal{T}\}$;
      $pre(\mathcal{T}) \leftarrow \bigwedge_{\gamma \in S_0^{\mathcal{T}}} \gamma \wedge \bigwedge_{\beta \in I^{\mathcal{T}}} \beta$;
    end
    $DNF(K^+(\alpha)) = \bigvee_{\mathcal{T} \in K^+} pre(\mathcal{T})$;
    $DNF(K^-(\alpha)) = \bigvee_{\mathcal{T} \in K^-} pre(\mathcal{T})$;
    Simplify $DNF(K^+(\alpha))$ and $DNF(K^+(\alpha))$;
    Add to nuSMV module:
    $next(\alpha) =$
    $case$
    $DNF(K^+(\alpha)) : TRUE;$
    $DNF(K^-(\alpha)) : FALSE;$
    $esac$
  end
end

Notice that the learning engine does not necessarily take into consideration the existence of groups of variables when performing the model adaptation. As a result, after learning, in the case of scalar variables, there may exist transitions associated with two or more different values of the

same variable. An analysis of this situation allows one to consider different possibilities such as (i) taking the largest value (i.e. the most activated neuron) or (ii) taking a range of possible values (essentially a disjunction in the head of a clause). This will be discussed in more detail in the light of an example below. Briefly, while the original representation might impose determinism, the learning process can break this constraint, allowing different associations to be made. It is the job of the knowledge extraction method to consider the scalar variables associated with the transitions so that the extracted knowledge is appropriate to the representation intended. The example in the following section will further discuss both cases (i) and (ii), illustrating in detail how option (ii) can be used to model nondeterministic situations.

In case (i) above, basically determinism is imposed onto the learning process; the extracted transitions will assign only one of the variables to *true* for each group, and the weight of a transition will reflect the comparison among the numerical values assigned to the variables in that group. At the end of this process, writing the transitions into NuSMV descriptions becomes identical to the binary case. In case (ii), all transitions are considered together with a probabilistic interpretation of the numerical weights. In this case, when writing the transitions into NuSMV, the different possibilities need to be analyzed in conjunction with the above-mentioned confidence value, as detailed below.

Finally, given a partial specification, NuSMV is capable of verifying the satisfiability of properties expressed in either CTL or LTL. If the specification does not satisfy the properties, a counter-example is produced. A counter-example is given by a sequence of assignments to variables, illustrating a situation where a property is not satisfied. In other words, a counter-example is very similar to the examples in Table II, only they are undesirable and the network should learn to avoid them. This information can be as useful as training examples in guiding the learning process. In particular, any small change made to the target output of a counter-example should be sufficient to move the network away from that particular undesired state (since the network seeks to maintain consistency with its training examples). Hence, turning a counter-example into a potentially useful training example should be straightforward. A somewhat more difficult question, however, is how to produce *good* training examples. In this paper we deliberately avoid this issue, although we are aware of related work that might be useful in connection with our framework, and will return to the issue in the future work section. Since we are interested in evaluating the viability of the framework, we chose to generate simple possible training examples by making small changes to the final transition of the sequence, and running the framework as many times as necessary. In general, we assume that an expert can choose which elements of the counter-example should be kept or changed when defining a learning sequence from that counter-example.

4. EXPERIMENTAL EVALUATION AND RESULTS

In this section, we apply and evaluate the V&A framework on the benchmark *pump system* example, considering both black box learning (i.e. learning from system runs only) and specification evolution (i.e. to achieve property satisfaction). We also apply the framework to a small *switch* example to illustrate its ability to learn and extract nondeterministic knowledge. Finally, we investigate the framework's utility by deploying it, with the help of a domain expert, on a real case study about fault diagnosis in a power plant. The results show that the V&A framework can be useful at achieving adaptation and property approximation, and help experts in the interactive process of knowledge discovery and requirements elicitation, as detailed below.

4.1. Black Box Learning: The Pump System

First, we investigate the question of learning a specification given only the observed behaviour of an existing system. We use the *pump system* case study [Alrajeh et al. 2009a; Alrajeh et al. 2009b] for this, as described as follows. The pump system monitors and controls the levels of water in a mine to avoid the risk of overflow, through the use of three state variables: $CrMeth$ indicates that the level of methane is critical, $HiWat$ indicates a high level of water, and $PumpOn$ indicates that the pump is turned on. In order to turn on and off such indicators, six different input signals are used:

Table III. NuSMV Description of the Pump System

```
MODULE PumpSystem
IVAR
s : {sCMOn, sCMOff, sHiW, sLoW, , TurnPOff};
VAR
CrMeth : boolean;
HiWat : boolean;
PumpOn : boolean;
ASSIGN
init(CrMeth) := FALSE;
init(HiWat) := FALSE;
init(PumpOn):= FALSE;
next(CrMeth) :=
case
s = sCMOn : TRUE;
s = sCMOff : FALSE;
esac;
next(HiWat) :=
case
s = sHiW : TRUE;
s = sLoW : FALSE;
esac;
next(PumpOn) :=
case
s = TurnPOn : TRUE;
s = TurnPOff : FALSE;
esac;
```

$sCMOn$, $sCMOff$, $sHiW$, $sLoW$, $TurnPOn$ and $TurnPOff$. Table III contains an initial NuSMV description of the system.

By applying the algorithms to translate the NuSMV description of the pump system into an SCTL network, first we obtain a description $\mathcal{P} = \left\{St^{\mathcal{P}}, In^{\mathcal{P}}, Init^{\mathcal{P}}, C^{\mathcal{P}}\right\}$ in the form of a temporal model, where the information in $C^{\mathcal{P}}$ is read from the *NextBlock* in the description, and the remaining information is given as follows, where $\sim$ stands for negation (below, we denote by $Gr^{\mathcal{P}}$ the groups of variables used to represent the scalar input variable).

$St^{\mathcal{P}} = \{CrMeth, HiWat, PumpOn\}$
$In^{\mathcal{P}} = \{sCMOn, sCMOff, sHiW, sLoW, TurnPOn, TurnPOff\}$
$Init^{\mathcal{P}} = \{\sim CrMeth, \sim HiWat, \sim PumpOn\}$
$Gr^{\mathcal{P}} = \{sCMOn, sCMOff, sHiW, sLoW, TurnPOn, TurnPOff\}.$
$C^{\mathcal{P}} = \{$
$\bigcirc CrMeth \leftarrow sCMOn$
$\bigcirc CrMeth \leftarrow CrMeth, \sim sCMOff$
$\bigcirc HiWat \leftarrow sHiW$
$\bigcirc HiWat \leftarrow CrMeth, \sim sLoW$
$\bigcirc PumpOn \leftarrow TurnPOn$
$\bigcirc PumpOn \leftarrow CrMeth, \sim TurnPOff$
$\}$

We can either translate the above temporal rules directly into an SCTL network or train a network to learn the behaviour described in Table III from examples. In this section, we are interested in investigating this learning task, which we call *black-box learning*. To evaluate this task, we assume that the behaviour provided by the temporal rules is the desired behaviour of the system to be learned. We have, thus, used the rules to produce observed examples, recording random inputs and their associated state values (as given by the rules) for 1000 timepoints. This set of observed examples was then presented to the tool implementation of our V&A framework without background knowledge (i.e. to train a network initially set with random weights). With this experiment, we are

interested in verifying if the behaviour of the system (as given by the rules) can be learned from scratch, by examples only. Then, we considered sets of examples with errors (i.e. observed examples that do not reflect the actual desired behaviour of the system) at performing the same task. This corresponds to situations that can be caused by errors in the original observed system, or by problems when recording the observations. This time round, when generating observed examples, we have changed the state values, according to a pre-defined percentage of error, to perform transitions to a random state, instead of the desired state as given by the rules.

In order to evaluate robustness of the learning process in the presence of errors, we have measured, as usual, the network's accuracy at learning, but also the network's *confidence* in its outputs, defined as: $\sqrt{\sum_i out_i^2/n}$, for every output value $out_i (1 \leq i \leq n)$. The closer the output is to either 1 (true) or -1 (false), the higher the confidence; network outputs around zero indicate a high level of uncertainty. Figure 3 shows the results starting with 0% errors up to 50% errors in the set of examples (i.e. when half of the transitions lead to a random state), with intervals of 5%. The results show *graceful degradation* (as oppposed to a sudden drop in performance) in the presence of increasing noise (number of errors).

Qualitatively, we have also sought to compare the extracted model after training with the original set of rules from which the examples were derived. The extracted model corresponded to the original description in Table III up until 30% error was added. At 35% error and above, the extracted model started to differ from the original model.

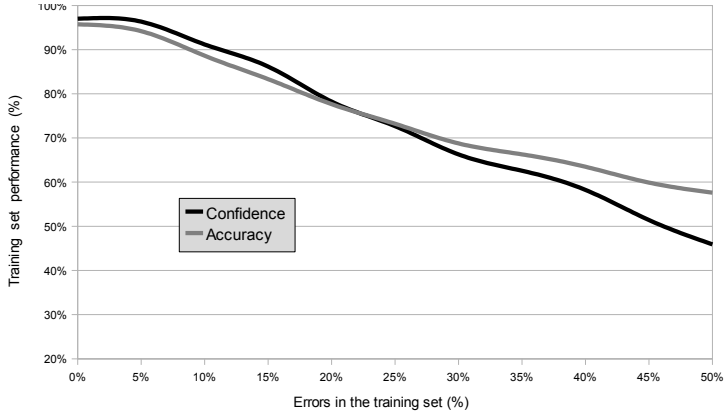

Fig. 3. Accuracy and Confidence of Model with Errors

The above results indicate the robustness of the network approach. Our framework has managed to learn the desired behaviour of a system with up to a 30% level of errors in the observations. As expected, increases in the level of errors have led to a lower confidence of the framework, but not to complete (or abrupt) failure. A confidence threshold of 65% was associated with the error boundary at which the extracted learned models started to differ in behaviour from the original, desired specification.

4.2. Evolving Specifications

In this experiment, we investigate the integration of verification and adaptation within the V&A framework. We consider the same description of the Pump System above, only now together with a safety property, expressed in LTL as follows:

$$G\neg(CrMeth \wedge HiWat \wedge PumpOn),$$

Table IV. Initial Counter-example Sequence

Initial Counter-example		
t	State	Input
1	{}	$s = sCMon$
2	$\{CrMeth\}$	$s = sHiW$
3	$\{CrMeth, HiWat\}$	$s = turnPon$
4	$\{CrMeth, HiWat, PumpOn\}$	–

meaning that the pump should not be *on* when the level of methane is critical and the water is high at the same time. Table IV shows the counter-example retrieved after the model verification of this property by NuSMV, indicating that the property is not satisfied by the current specification.

As discussed earlier, a simple way of turning the above counter-example into a learning sequence is to flip the truth-value of one of the variables in the final state of the counter-example, e.g. *PumpOn*, and make the sequence more general by assigning zero to the other variables. In this experiment, this procedure produces the following learning sequence: {} $\rightarrow$ $sCMOn \rightarrow sHiW \rightarrow TurnPOn \rightarrow \{\neg PumpOn\}$. Given a learning sequence and translation of the temporal rules into an initial network, the adaptation process can start. Notice that learning is independent of the syntactic structure of the original rules; therefore a different structure altogether can be generated by the network in order to unify the original knowledge and the learning examples. For completeness, we also considered the network obtained by learning from examples in the previous section as an alternative initial network. Both networks were submitted to different configurations of the learning process and parameters; the networks' set-up is available, together with our tool, at http://vega.soi.city.ac.uk/~abct616/?cont=2. After training, the extraction algorithm was applied and state transition diagrams were produced, showing two learned hypotheses: in Fig. 4(a), the only situation where the pump switches from off to *on* is when both $CrMeth$ and $HiWat$ are $false$. In the figure, M is used to denote that $CrMeth$ is *true*, W to denote that $HiWat$ is *true*, and P that *PumpOn* is *true*. The diagram shows all (eight) combinations of truth-values for M, W, P and relevant transitions; M, W, P are not shown in the diagram when their truth-value is $false$. In Fig. 4(b), however, $CrMeth$ can lead to *PumpOn* by itself, and so can $HiWat$. In both cases, as expected the counter-example no longer holds after training, since no transition exists from M, W to M, W, P.

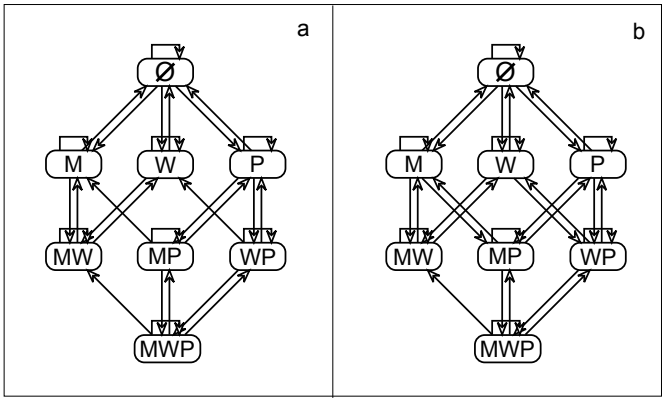

Fig. 4. Transition Diagrams after Property Adaptation

Let us take the network associated with Fig. 4(b) to continue our analysis. One can represent the trained network as a new NuSMV model. This is shown in Table V. If we compare this new description with the original one from Table III, it can be seen that a new condition was added to the case where *PumpOn*. This learned condition is more general than simply disallowing the counter-example, making this learning process useful. However, there are still transitions in the diagram

Table V. Adapted NuSMV description

```
next(CrMeth) :=
case
s = sCMOn : TRUE;
s = sCMOff : FALSE;
esac;
next(HiWat) :=
case
s = sHiW : TRUE;
s = sLoW : FALSE;
esac;
next(PumpOn) :=
case
!CrMeth & (s = TurnPOn) : TRUE;
!HiWat & (s = TurnPOn) : TRUE;
s = TurnPOff : FALSE;
esac;
```

Table VI. New Counter-example Sequence

New Counter-example		
t	State	Input
1	$\{\sim CrMeth, \sim HiWat \sim PumpOn\}$	$sCMOn$
2	$\{CrMeth, \sim HiWat, \sim PumpOn\}$	$TurnPOn$
3	$\{CrMeth, \sim HiWat, PumpOn\}$	$sHiW$
4	$\{CrMeth, HiWat, PumpOn\}$	–

Table VII. Final Counter-example Sequence

Final Counter-example		
t	State	Input
1	$\{\sim CrMeth, \sim HiWat \sim PumpOn\}$	$sHiW$
2	$\{\sim CrMeth, HiWat, \sim PumpOn\}$	$TurnPOn$
3	$\{\sim CrMeth, HiWat, PumpOn\}$	$sCrMeth$
4	$\{CrMeth, HiWat, PumpOn\}$	–

leading to M, W, P and a violation of the property. This can be verified formally by repeating the V&A process, now on the new NuSMV model of Table V. Following [d'Avila Garcez et al. 2001], the idea here is to repeat this interactive cycle of verification and adaptation, guided by the above generalisations over counter-examples, until our property is satisfied. In this experiment, if we try to verify our property on the new NuSMV model, a new counter-example, shown in Table VI, will be created.

Given the counter-example of Table VI, a new learning sequence can be created, as follows: $\{\sim CrMeth, \sim HiWat \sim PumpOn\} \rightarrow sCMOn \rightarrow sHiW \rightarrow turnPon \rightarrow \{\neg PumpOn\}$. As before, we then considered different training parameters. The diagram of Fig. 5(a) shows the extracted learned model. It is not difficult to see that our property is still not satisfied. We then expressed the model again in the NuSMV language, and performed verification, obtaining a further counter-example (shown in Table VII). This was again turned into a learning sequence, followed by training and extraction of the diagram in Fig. 5(b) (expressed in the form of a new NuSMV description in Table VIII). When applying the model checker to this new description, the property is finally satisfied (as the state diagram makes clear).

The above experiment illustrates the V&A cycle and indicates its usefulness as an interactive evolution process. Next, we conduct a different experiment to investigate the representation and learning (from examples) of scalar variables, and apply the V&A tool, given the above results, to a real case study.

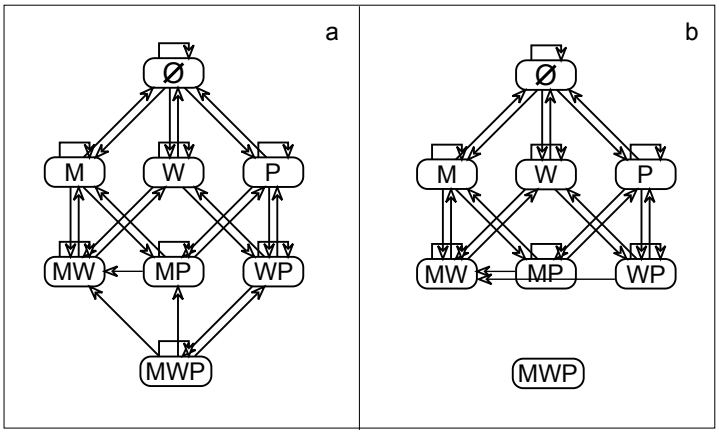

Fig. 5. New Transition Diagrams Extracted

Table VIII. Final NuSMV description satisfying the property

```
next(CrMeth) :=
case
s = sCMOn : TRUE;
s = sCMOff : FALSE;
esac;
next(HiWat) :=
case
s = sHiW : TRUE;
s = sLoW : FALSE;
esac;
next(PumpOn) :=
case
!CrMeth & (s = TurnPOn) : TRUE;
!HiWat & (s = TurnPOn) : TRUE;
CrMeth & PumpOn & (s = sHiW) : FALSE;
HiWat & PumpOn & (s = sCMon) : FALSE;
s = TurnPOff : FALSE;
esac;
```

4.3. Learning Non-deterministic Knowledge

Let us now consider a simple non-deterministic system which can assume three different states: *left, centre* or *right*. The system also has a *switch*: an input that, when activated, changes the state to a neighbouring position, that is, if the system is at the *left* state it cannot go directly to the *right* state, and vice-versa. If the system is at the *centre* state, it can go either *left* or *right*; let p denote the probability of the system going *left* from *centre*.

In this experiment, the V&A tool *observes* the system given different values for p. Our goal is to evaluate the tool's adaptation process in the presence of non-determinism and scalar variables. Table IX shows the extracted transitions after the learning phase took place in the network (left column) and the confidence values of each transition in the cases where $p = 0, 0.25, 0.5, 0.75$ and 1. The network has input variable *switch*, and the state variables $left$, *centre* and *right* (represented, respectively, by L, C and R, in the table). Learning took place by observation only, i.e. without an initial description of the model being added to the network.

As can be seen, in the deterministic cases ($p = 0$ and $p = 1$), the network behaves as before, with high confidence in all of the transitions. In the case where $p = 0.25$ (resp. $p = 0.75$), the network treats the variation as noise, assuming the correct transition to be from *centre* to *left* (resp. *right*) but with a lower confidence, around 84% (resp. 79%). The interesting new result from this experiment

Table IX. Extracted transitions in the presence of non-determinism

	p = ...				
Transitions	0%	25%	50%	75%	100%
$\{L\} \rightarrow no\_switch \rightarrow \{L\}$		99%	94%	99%	100%
$\{C\} \rightarrow no\_switch \rightarrow \{C\}$	100%	100%	98%	99%	100%
$\{R\} \rightarrow no\_switch \rightarrow \{R\}$	100%	100%	95%	99%	
$\{L\} \rightarrow switch \rightarrow \{C\}$		100%	98%	99%	99%
$\{R\} \rightarrow switch \rightarrow \{C\}$	100%	100%	97%	99%	
$\{C\} \rightarrow switch \rightarrow \{L\}$			43%	79%	100%
$\{C\} \rightarrow switch \rightarrow \{R\}$	100%	84%	43%		
$\{C\} \rightarrow switch \rightarrow \{L, R\}$			44%		
$\{C\} \rightarrow switch \rightarrow \emptyset$			44%		

occurs, as expected, when $p = 0.5$; in this case, Table IX shows four possible transitions when *switch* is *true* and *state* = *centre*. The four possible outcomes, represented by L, R, L, R and $\emptyset$ in the table, refer to *state* = $left$, *state* = *right*, *both* (i.e. both $left$ and *right* outputs are *true*) and *neither* (i.e. no network output is *true*). Despite L, R and $\emptyset$ being impossible, each of the four transitions had a confidence of around 44%. This is because no constraints were imposed on the network, e.g. in the form of a property.

This experiment's results exemplifies the options available to us in the V&A framework, as discussed earlier. The first is to explicitly constrain the learning process so that, e.g., transitions like $\{C\} \rightarrow switch \rightarrow \{L, R\}$ and $\{C\} \rightarrow switch \rightarrow \emptyset$ are disallowed. As shown by the experiments, the network's own noise tolerance has dealt satisfactorily with most of the cases, without the need for explicit constraints. However, in the case of maximum uncertainty (i.e. $p = 0.5$), this could not have been expected, and further information would have been needed to decide on the outcome, as indicated by the low confidence levels. The second option is to treat this low confidence levels themselves as an indication of non-determinism in the system being observed. Even though it is impossible to decide between $left$ and *right* in the case of $p = 0.5$, it is interesting noting that the network has managed to learn that the system should not be allowed to stay in a state when *switch* is triggered. For examples, based on the training examples only, the network has learned that the transition $\{C\} \rightarrow switch \rightarrow \{C\}$ was undesirable. The network was unable to decide between $left$ or *right*, as expected since $p = 0.5$, but it was able to learn from the examples that no change was not an option. This is evidence of the generalization capacity of these networks and their ability to capture non-determinism in the observed system.

Given the intractability of the verification problem, our objective has been to check if our tool can be useful in specific applications. Using the above results as *proof-of-concept*, it remains to be seen if the V&A framework can be useful in a real application. In the next section, we answer this question positively in the case of a fault diagnosis case study in a power plant. The results show that it is possible to evolve error-prone specifications as part of an interactive process of verification and adaptation towards a property-satisfying specification.

4.4. Case Study: Power Plant Fault Diagnosis

We have applied the V&A framework to a case study in the area of fault diagnosis using a simplified version of a real power generator plant. The task at hand is to quickly identify possible faults in the power plant when a given set of alarms is triggered. The framework is expected to learn a mapping from a set of alarms (represented in the input layer of the network) to the set of possible faults (represented in the output layer of the network). In addition, the framework is expected to identify if certain faults can trigger some other, derived faults. The most general set-up, therefore, is that of a network representing alarms (input variables) and certain faults (state variables) in the input layer, and faults in the output layer with recurrent connections from the output to the faults in the input. In the case of a system failure, alarms might fail to activate due to equipment failure. We are, therefore, interested in investigating how well the networks perform at predicting faults in such error-prone situation (i.e. when alarms might fail to activate).

The case study (logical rules and training examples) was derived at the Research Center of the largest power company in Brazil (CEPEL, Eletrobras) by V.N. Silva (see [d'Avila Garcez et al. 1997]). Fig. 6 shows the power plant with two generators, two transformers with their respective circuit breakers, two buses (main and auxiliary), and two transmission lines and a bypass circuit, also with their respective circuit breakers.

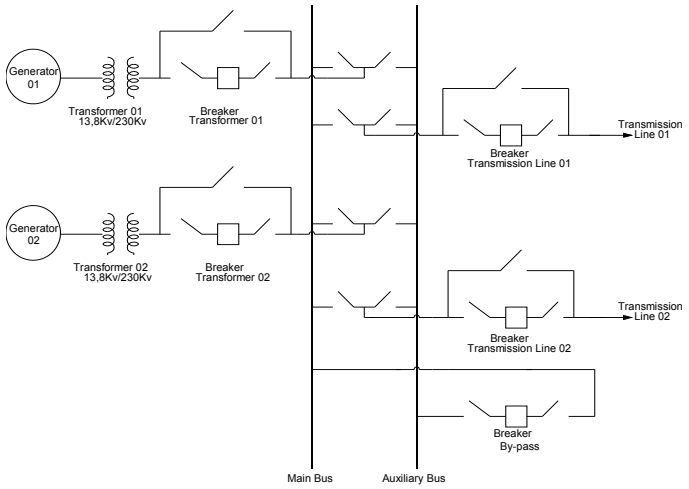

Fig. 6. Simplified model of power system generation plant

Each transmission line has six associated alarms: *breaker* status (indicates that the breaker is open), *phase* over-current (indicates an over-current in the phase line), *ground* over-current (indicates an over-current in the ground line), *timer* (indicates a fault distant from the power plant generator), *instantaneous* (indicates a fault close to the power plant generator), and *auxiliary* (indicates that the transmission line is connected to the auxiliary bus).

In addition, each transformer has three associated alarms: *breaker* status (indicates that the breaker is open), *overload* (indicates a transformer overload) and *auxiliary* (indicates that the transformer is connected to the auxiliary bus). Finally, there are five alarms associated with the bypass circuit breaker: *breaker* status, *phase* over-current, *ground* over-current, *timer* and *instantaneous*.

We describe the set of alarms by the predicate $Alarm(type, place)$, where $type \in \{breaker\_open, phase, ground, timer, instantaneous, auxiliary, overload\}$ and $place \in \{line01, line02, transformer01, transformer02, bypass\}$. Certain combinations of alarms indicate certain faults in the system. They indicate the *component* affected by the fault (whether a transmission line or a transformer), the *reason* for the fault (if it is a phase or ground over-current, a problem in the main or auxiliary bus, or a transformer overload), and the *proximity* of the fault to the power plant generator. In addition, some faults indicate whether the system had been using the *bypass* circuit when an alarm was activated.

We describe the set of faults by the predicate

$$Fault(occurrence, proximity, component, bypass\_use)$$

s.t. $occurrence \in \{phase, ground, main\_bus, auxiliary\_bus, overload\}$, $proximity \in \{close\text{-}up, distant\}$, $component \in \{line01, line02, transformer01, transformer02\}$, and $bypass\_use \in \{no\_bypass, bypass\}$.

In the above electric system, it is known that if the Instantaneous alarm of Transmission Line 01 is activated then there is a fault at Transmission Line 01 close up to the power plant generator. This relation can be described as follows:

$$Fault(x, close\_up, line01, no\_bypass) \leftarrow Alarm(instantaneous, line01)$$

where $x \in \{ground, phase\}$. If, in addition, the alarm that indicates an over-current in the ground line of Transmission Line 01 is activated then it is also known that the fault is due to a ground over-current. This relation is represented as:

$$Fault(ground, close\_up, line01, no\_bypass) \leftarrow$$
$$Alarm(instantaneous, line01), Alarm(ground, line01)$$

which reads "*there is a fault at transmission line 01, close to the power plant generator, due to an over-current in the ground line of transmission line 01, which occurred when the system was not using the bypass circuit*".

Finally, certain alarms also eliminate the possibility of certain faults occurring in the main bus or in each of the transformers. For example, if Transformer 02 is connected to the auxiliary bus then it is not the case that Transformer 02 could be overloaded.

The logical rules that are known to the engineer include (the rules map alarms to faults and thefefore do not contain a temporal operator; the relations between faults and derived faults is unknown and it is our goal to identify those as a result of learning):

— Twenty nine (instantiated) rules of the form:
$Fault(occurrence, proximity, component, bypass\_use) \leftarrow \bigwedge_i (Alarm(type, local))_i$;
— Four rules of the form:
$\neg Fault(main\_bus, proximity, component, bypass\_use) \leftarrow Alarm(auxiliary, x)$, where $x \in \{line01, line02, transformer01, transformer02\}$; and
— Two rules of the form:
$\neg Fault(overload, proximity, y, bypass\_use) \leftarrow Alarm(auxiliary, y)$, where $y \in \{transformer01, transformer02\}$.

In the complete rule set (listed in Appendix 1), 35 rules associate 23 different alarms with 32 different faults. As a result, a network with 55 input neurons (alarms and faults), 35 hidden neurons (one for each rule) and 32 output neurons (faults) was created. The faults in the output layer are recurrently connected to the faults in the input layer.

In addition, we have added errors to the background knowledge by changing the conditions of two rules randomly to wrong conditions: Rules 24 and 25 (see Appendix 1) were changed, respectively, into rules 24′ and 25′, as follows (below, we use the character "$*$" to indicate "don't care"):

24′ *Fault*(*main_bus*, $*$, *transformer01,* $*$) ← (*Alarm*(*breaker_open*, *transformer01*), *Alarm*(*breaker_open*, *bypass*), *Alarm*(*breaker_open*, *line01*), *Alarm*(*breaker_ open*, *line02*), *Alarm*(*breaker_open*, *transformer02*)).

25′ *Fault*(*main_bus*, $*$, *transformer02,* $*$) ← (*Alarm*(*breaker_open*, *transformer02*), *Alarm*(*breaker_open*, *bypass*), *Alarm*(*breaker_open*, *line01*), *Alarm*(*breaker_ open*, *line02*), *Alarm*(*breaker_open*, *transformer01*)).

in which *Alarm*(*auxiliary*, *transformer01*) and *Alarm*(*auxiliary*, *transformer02*) were replaced, respectively, by *Alarm*(*breaker_open*, *transformer01*) and *Alarm*(*breaker_ open*, *transformer02*).

In addition to the logical rules, the case study makes use of 278 examples which include single and multiple faults, i.e, examples where each set of alarms is associated with a unique possible fault, and examples where each set of alarms is associated with many possible faults.

Permanent System Properties:

Most faults may occur with or without the breaker by-pass being actuated, and those cannot happen at the same time. For example, faults 1 and 2 (see Appendix 1) can never happen at the same time. We express such a system property as follows, where G denotes always in the future:

$$G\neg(Fault(*,*,X,bypass) \wedge Fault(*,*,X,no\_bypass))$$

In addition, since there is a single by-pass in the power plant, it cannot replace at the same time the breakers in transmission lines 01 and 02. Hence:

$$G\neg(Fault(*,*,line01,bypass) \wedge Fault(*,*,line02,bypass))$$

First, we trained a network with a fixed sequence of examples, after adding the rules with errors as background knowledge. Figure 7 shows a state transition diagram extracted from the trained network. In Fig. 7 each state is labelled with a set of faults and each link with a set of inputs (alarms) that have triggered that transition; alarm and fault numbers are as listed in Appendix 1. For example, in the figure, if faults 2 and 4 are observed and no alarm is triggered then the network stays at that stable state (denoted by the reflexive link in node (2,4)). If, on the other hand, faults 2 and 4 are observed and alarm 17 is triggered then the network *narrows* the faults down to fault 4 only. If either alarm 16 or alarms 12 and 16 are triggered then the network no longer believes that a fault exists. These transition patterns are currently being analysed by a domain expert.

Notice how any combination of faults {2,4} with {10,12} imply a violation of the property stating that *line01* and *line02* must not occur together with *bypass*. States (2,10), (2,12), (4,10), (4,12) and (2,4,10), (2,4,12), (2,10,12), (4,10,12), as well as (2,4,10,12), all violate the property.

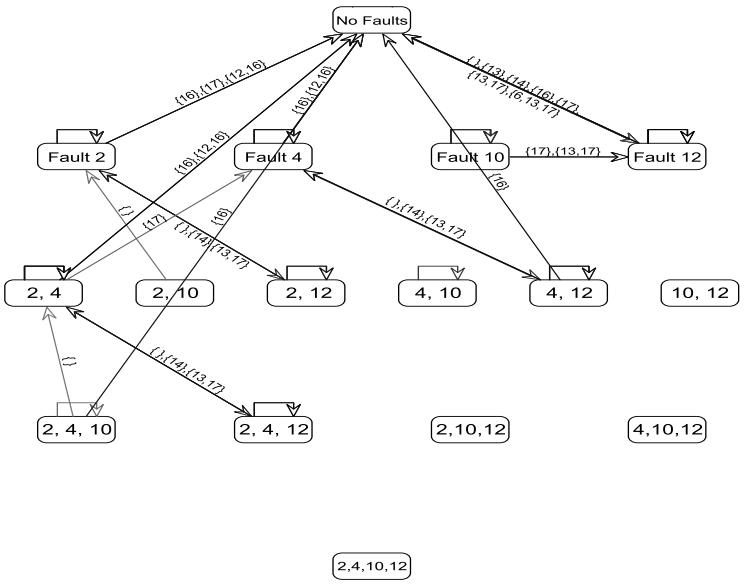

Fig. 7. Diagram extracted from a network trained with the power plant data

Then, we have trained the network further with a different sequence of examples. We did not make use of counter-examples since a relatively large number of examples is already available.

Figure 8 shows a state transition diagram extracted from the network that was trained further. Notice how, in Fig. 8, none of the above nodes that were violating the property in Fig. 7 are reachable. For instance, violation state (2,4,12), which is reachable in Fig. 7, is not reachable in Fig. 8, indicating that the V&A extraction and visualization method can be useful at monitoring the adaptation

process in real applications. This is an interesting development, since scalability of the extraction method was a concern.

It is also interesting noting that, in Fig. 8, state 12 seems to act as an *attractor* node. This is of interest to domain experts because it indicates how certain faults might be derived from other faults not previously identified. In this particular experiment, the V&A tool has identified a relationship *between faults* that had not been identified before by domain experts (notice how the logical rules and training examples only define relationships directly from alarms to faults). This can be useful to domain experts, since it provides a tool for analysis and the potential to uncover new knowledge. In this case of a model of a real power plant, new critical information might be discovered. Although further analysis of this case study is ongoing, the V&A framework has been shown useful in practice in providing new information to the specialist, and potentially useful even in the case of a relatively large network and real-world system.

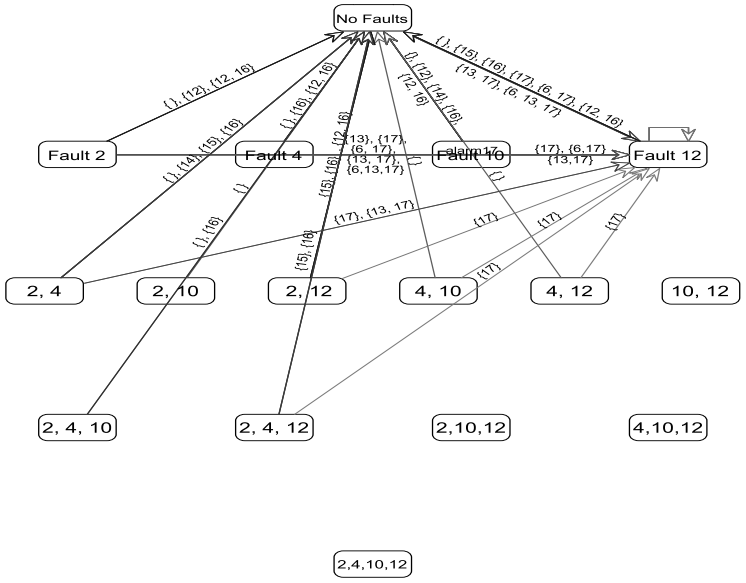

Fig. 8. Diagram extracted after further training on power plant data

In summary, the V&A framework can be useful at helping domain experts verify some important patterns that might emerge from an adaptation process. Further, the V&A framework can be effective as an adaptation tool in a real application even in the presence of errors in the background knowledge, as can be seen by comparing Figs. 7 and 8.

5. CONCLUSIONS AND FUTURE WORK

This paper proposed a new methodology for integrated learning and adaptation of evolving systems specifications. We have developed a novel framework and tool capable of verification and learning of temporal models. The framework allows not only the evolution of existing models guided by properties, but learning from observation of system runs without the need for an initial abstract description of its behaviour to be provided in a specific language. The model uses formal verification tools and machine learning in an integrated way, allowing error and inconsistency management in different steps of the software development process. The approach uses model checking for verifying properties against an initial model description. In addition, a machine learning tool is integrated with a model checker in order to adapt and evolve the model when a property is violated. An innovative feature of our work, with respect to related approaches, is its robustness and effective error

handling in the specification process. These features allow improved knowledge gathering about a system's required behaviour, as shown by experiments. The automatic process of evolving the model with respect to systems' properties can help domain experts in the task of system specification and analysis.

We have validated our framework on several case studies. Special attention was given to the evaluation of how well the framework copes with errors in specifications. The framework was implemented as a tool that is fully integrated with a model checker. The tool has been shown capable of integrated verification and evolution of abstract models, and also of re-engineering partial models given examples from existing systems. We have also illustrated that the learning system deals with uncertainty and errors, and is capable of capturing the non-deterministic nature of certain systems. Finally, we have applied the framework and tool to a real power plant case study; results were analysed by a domain expert and have shown that knowledge descriptions can be discovered by this process, even in the presence of errors and in a real-world application based on a relatively large network model. Extensions of the formalism to represent first-order models and properties may increase the applicability of the framework. Further, the application of the framework to other real-life testbeds would be welcome, so that one could verify performance in different application domains. The evaluation of the framework is ongoing.

The V&A methodology could also be useful in product-focused software process improvement [Nuseibeh 2010], in particular in embedded, autonomous and error-prone software systems. These systems increasingly affect peoples lives and their safe deployment demands sound software engineering approaches to their development.

Recent developments in autonomous systems demand new tools and theories capable of offering answers to such systems' growing complexity and needs [Dobson et al. 2006; Dobson et al. 2010]. Autonomous systems demand a new reasoning and systems engineering paradigm, since one has to cater for the so-called self-* objectives or "broad requirements" [Dobson et al. 2010]: self-configuring, self-healing, self-optimizing and self-protecting, which are met by attributes like self-awareness, self-monitoring, etc. Formal methods research in general, and model checking, in particular will, therefore, require the development of associated methods and tools in order to respond to such challenges [Clarke et al. 2009; Dobson et al. 2006; Dobson et al. 2010; Parnas 2010].

Another extension would include the explicit use of probabilities and a comparison, perhaps, with probabilistic symbolic approaches [Raedt et al. 2008]. While a direct comparison would be welcome, and there are efforts, only now starting, to integrate inductive logic programming with neural-symbolic systems [Franca et al. 2012], neural networks have been shown generally superior at robust, error-prone learning [d'Avila Garcez et al. 2002; Uwents et al. 2011]. Hence, our starting point in this paper has been to use mainstream learning methods in software engineering, supported by a methodology and framework that allow the appropriate use of both machine learning and verification in a tool for effectively evolving software specifications.

6. APPENDIX 1: RULE SET FOR THE POWER PLANT CASE STUDY

(1) *Fault*(*phase*, *close-up*, *line01*, *no_bypass*) ← (*Alarm*(*phase*, *line01*), *Alarm* (*instantaneous*, *line01*)).
(2) *Fault*(*phase*, *close-up*, *line01, bypass*) ← (*Alarm*(*auxiliary*, *line01*), *Alarm*(*phase*, *bypass*), *Alarm*(*instantaneous*, *bypass*)).
(3) *Fault*(*ground*, *close-up*, *line01*, *no_bypass*) ← (*Alarm*(*ground*, *line01*), *Alarm* (*instantaneous*, *line01*)).
(4) *Fault*(*ground*, *close-up*, *line01, bypass*) ← (*Alarm*(*auxiliary*, *line01*), *Alarm* (*ground*, *bypass*), *Alarm*(*instantaneous*, *bypass*)).
(5) *Fault*(*phase*, *distant*, *line01*, *no_bypass*) ← (*Alarm*(*phase*, *line01*), *Alarm* (*timer*, *line01*)).
(6) *Fault*(*phase*, *distant*, *line01, bypass*) ← (*Alarm*(*auxiliary*, *line01*), *Alarm* (*phase*, *bypass*), *Alarm*(*timer*, *bypass*)).
(7) *Fault*(*ground*, *distant*, *line01*, *no_bypass*) ← (*Alarm*(*ground*, *line01*), *Alarm* (*timer*, *line01*)).

(8) *Fault*(*ground*, *distant*, *line01, bypass*) ← (*Alarm*(*auxiliary*, *line01*), *Alarm* (*ground*, *bypass*), *Alarm*(*timer*, *bypass*)).
(9) *Fault*(*phase*, *close-up*, *line02*, *no_bypass*) ← (*Alarm*(*phase*, *line02*), *Alarm* (*instantaneous*, *line02*)).
(10) *Fault*(*phase*, *close-up*, *line02, bypass*) ← (*Alarm*(*auxiliary*, *line02*), *Alarm*(*phase*, *bypass*), *Alarm*(*instantaneous*, *bypass*)).
(11) *Fault*(*ground*, *close-up*, *line02*, *no_bypass*) ← (*Alarm*(*ground*, *line02*), *Alarm* (*instantaneous*, *line02*)).
(12) *Fault*(*ground*, *close-up*, *line02, bypass*) ← (*Alarm*(*auxiliary*, *line02*), *Alarm* (*ground*, *bypass*), *Alarm*(*instantaneous*, *bypass*)).
(13) *Fault*(*phase*, *distant*, *line02*, *no_bypass*) ← (*Alarm*(*phase*, *line02*), *Alarm* (*timer*, *line02*)).
(14) *Fault*(*phase*, *distant*, *line02, bypass*) ← (*Alarm*(*auxiliary*, *line02)*, *Alarm*(*phase*, *bypass*), *Alarm* (*timer*, *bypass*)).
(15) *Fault*(*ground*, *distant*, *line02*, *no_bypass*) ← (*Alarm*(*ground*, *line02*), *Alarm* (*timer*, *line02*)).
(16) *Fault*(*ground*, *distant*, *line02, bypass*) ← (*Alarm*(*auxiliary*, *line02*), *Alarm* (*ground*, *bypass*), *Alarm*(*timer*, *bypass*)).
(17) ¬*Fault*(*main_bus*, ∗, ∗, ∗) ← *Alarm*(*auxiliary*, *line01*).
(18) ¬*Fault*(*main_bus*, ∗, ∗, ∗) ← *Alarm*(*auxiliary*, *line02*).
(19) ¬*Fault*(*main_bus*, ∗, ∗, ∗) ← *Alarm*(*auxiliary*, *transformer01*).
(20) ¬*Fault*(*main_bus*, ∗, ∗, ∗) ← *Alarm*(*auxiliary*, *transformer02*).
(21) *Fault*(*main_bus*, ∗, ∗, ∗) ← (*Alarm*(*breaker_open*, *line01*), *Alarm* (*breaker_ open*, *line02*), *Alarm*(*breaker_open*, *transformer01*), *Alarm*(*breaker_open*, *transformer02*)).
(22) *Fault*(*main_bus*, ∗, *line01,* ∗) ← (*Alarm*(*auxiliary*, *line01*), *Alarm*(*breaker_ open*, *bypass*), *Alarm*(*breaker_open*, *line02*), *Alarm*(*breaker_open*, *transformer 01*), *Alarm*(*breaker_open*, *transformer02*)).
(23) *Fault*(*main_bus*, ∗, *line02,* ∗) ← (*Alarm*(*auxiliary*, *line02*), *Alarm*(*breaker_ open*, *bypass*), *Alarm*(*breaker_open*, *line01*), *Alarm*(*breaker_open*, *transformer 01*), *Alarm*(*breaker_open*, *transformer02*)).
(24) *Fault*(*main_bus*, ∗, *transformer01,* ∗) ← (*Alarm*(*auxiliary*, *transformer01*), *Alarm*(*breaker_open*, *bypass*), *Alarm*(*breaker_open*, *line01*), *Alarm* (*breaker_ open*, *line02*), *Alarm*(*breaker_open*, *transformer02*)).
(25) *Fault*(*main_bus*, ∗, *transformer02,* ∗) ← (*Alarm*(*auxiliary*, *transformer02*), *Alarm*(*breaker_ open*, *bypass*), *Alarm*(*breaker_open*, *line01*), *Alarm* (*breaker_ open*, *line02*), *Alarm*(*breaker_open*, *transformer01*)).
(26) *Fault*(*auxiliary_bus*, ∗, *line01, bypass*) ← (*Alarm*(*auxiliary*, *line01*), *Alarm* (*breaker_open*, *bypass*)).
(27) *Fault*(*auxiliary_bus*, ∗, *line02, bypass*) ← (*Alarm*(*auxiliary*, *line02*), *Alarm* (*breaker_open*, *bypass*)).
(28) *Fault*(*auxiliary_bus*, ∗, *transformer01, bypass*) ← (*Alarm*(*auxiliary*, *transformer01*), *Alarm*(*breaker_open*, *bypass*)).
(29) *Fault*(*auxiliary_bus*, ∗, *transformer02, bypass*) ← (*Alarm*(*auxiliary*, *transformer02*), *Alarm*(*breaker_open*, *bypass*)).
(30) ¬*Fault*(*overload*, ∗, *transformer01*, *no_bypass*) ← *Alarm*(*auxiliary*, *transformer01*).
(31) *Fault*(*overload*, ∗, *transformer01*, *no_bypass*) ← *Alarm*(*overload*, *transformer01*).
(32) ¬*Fault*(*overload*, ∗, *transformer02*, *no_bypass*) ← *Alarm*(*auxiliary*, *transformer02*).
(33) *Fault*(*overload*, ∗, *transformer02*, *no_bypass*) ← *Alarm*(*overload*, *transformer02*).
(34) *Fault*(*overload*, ∗, *transformer01, bypass*) ← (*Alarm*(*overload*, *transformer01*), *Alarm*(*auxiliary*, *transformer01*)).
(35) *Fault*(*overload*, ∗, *transformer02, bypass*) ← (*Alarm*(*overload*, *transformer02*), *Alarm*(*auxiliary*, *transformer02*)).

7. APPENDIX 2: LIST OF INPUT (ALARMS AND FAULTS) AND OUTPUT NEURONS (FAULTS)

(1) *Alarm*(*breaker_open*, *line01*)
(2) *Alarm*(*phase*, *line01*)
(3) *Alarm*(*ground*, *line01*)
(4) *Alarm*(*timer*, *line01*)
(5) *Alarm*(*instantaneous*, *line01*)
(6) *Alarm*(*auxiliary*, *line01*)
(7) *Alarm*(*breaker_open*, *line02*)
(8) *Alarm*(*phase*, *line02*)
(9) *Alarm*(*ground*, *line02*)
(10) *Alarm*(*timer*, *line02*)
(11) *Alarm*(*instantaneous*, *line02*)
(12) *Alarm*(*auxiliary*, *line02*)
(13) *Alarm*(*breaker_open*, *bypass*)
(14) *Alarm*(*phase*, *bypass*)
(15) *Alarm*(*ground*, *bypass*)
(16) *Alarm*(*timer*, *bypass*)
(17) *Alarm*(*instantaneous*, *bypass*)
(18) *Alarm*(*breaker_open*, *transformer01*)
(19) *Alarm*(*auxiliary*, *transformer01*)
(20) *Alarm*(*overload*, *transformer01*)
(21) *Alarm*(*breaker_open*, *transformer02*)
(22) *Alarm*(*auxiliary*, *transformer02*)
(23) *Alarm*(*overload*, *transformer02*)

(1) *Fault*(*phase*, *close-up*, *line01*, *no_bypass*)
(2) *Fault*(*phase*, *close-up*, *line01, bypass*)
(3) *Fault*(*ground*, *close-up*, *line01*, *no_bypass*)
(4) *Fault*(*ground*, *close-up*, *line01, bypass*)
(5) *Fault*(*phase*, *distant*, *line01*, *no_bypass*)
(6) *Fault*(*phase*, *distant*, *line01, bypass*)
(7) *Fault*(*ground*, *distant*, *line01*, *no_bypass*)
(8) *Fault*(*ground*, *distant*, *line01, bypass*)
(9) *Fault*(*phase*, *close-up*, *line02*, *no_bypass*)
(10) *Fault*(*phase*, *close-up*, *line02, bypass*)
(11) *Fault*(*ground*, *close-up*, *line02*, *no_bypass*)
(12) *Fault*(*ground*, *close-up*, *line02, bypass*)
(13) *Fault*(*phase*, *distant*, *line02*, *no_bypass*)
(14) *Fault*(*phase*, *distant*, *line02, bypass*)
(15) *Fault*(*ground*, *distant*, *line02*, *no_bypass*)
(16) *Fault*(*ground*, *distant*, *line02, bypass*)
(17) *Fault*(*main_bus*, $*$, $*$, $*$)
(18) *Fault*(*main_bus*, $*$, *line01,* $*$)
(19) *Fault*(*main_bus*, $*$, *line02,* $*$)
(20) *Fault*(*main_bus*, $*$, *transformer01,* $*$)
(21) *Fault*(*main_bus*, $*$, *transformer02,* $*$)
(22) *Fault*(*auxiliary_bus*, $*$, *line01, bypass*)
(23) *Fault*(*auxiliary_bus*, $*$, *line02, bypass*)
(24) *Fault*(*auxiliary_bus*, $*$, *transformer01, bypass*)
(25) *Fault*(*auxiliary_bus*, $*$, *transformer02, bypass*)
(26) *Fault*(*overload*, $*$, *transformer01*, *no_bypass*)
(27) *Fault*(*overload*, $*$, *transformer02*, *no_bypass*)

(28) *Fault*(*overload*, ∗, *transformer01, bypass*)
(29) *Fault*(*overload*, ∗, *transformer02, bypass*)
(30) ¬*Fault(main_bus, ∗, ∗, ∗)*
(31) ¬*Fault(overload, ∗, transformer01, no_bypass)*
(32) ¬*Fault(overload, ∗, transformer02, no_bypass)*

ACKNOWLEDGMENTS

We are grateful to Vitor Silva (Eletrobras) and Gerson Zaverucha (UFRJ) for providing the power plant data and giving their time and domain expertise to analyse our results, and to Jeff Kramer, Alessandra Russo (Imperial College) and Christos Kloukinas (City Univ. London) for many discussions, their comments and feedback on this research.

REFERENCES

Alrajeh, D., Kramer, J., Russo, A., and Uchitel, S. 2009a. Learning operational requirements from goal models. In *ICSE 2009*. 265–275.

Alrajeh, D., Kramer, J., Russo, A., and Uchitel, S. 2012. Elaborating requirements using model checking and inductive learning. *IEEE Transactions on Software Engineering 99*.

Alrajeh, D., Ray, O., Russo, A., and Uchitel, S. 2009b. Using abduction and induction for operational requirements elaboration. *Journal of Applied Logic 7,* 3, 275–288.

Andrews, R., Diederich, J., and Tickle, A. 1995. A survey and critique of techniques for extracting rules from trained artificial neural networks. *Knowledge-based Systems 8*, 373–389.

Beer, I., Ben-David, S., Chockler, H., Orni, A., and Trefler, R. J. 2009. Explaining counterexamples using causality. In *CAV*.

Ben-Ari, M., Manna, Z., and Pnueli, A. 1981. The temporal logic of branching time. In *POPL '81*. ACM, 164–176.

Beyer, D., Henzinger, T., Jhala, R., and Majumdar, R. 2007. The software model checker Blast: Applications to software engineering. *Intl. J. Soft. Tools Tech. Tran. 9*, 505–525.

Bobaru, M. G., Pasareanu, C. S., and Giannakopoulou, D. 2008. Automated assume-guarantee reasoning by abstraction refinement. In *CAV*. 135–148.

Borges, R., d'Avila Garcez, A., Lamb, L. C., and Nuseibeh, B. 2011a. Learning to adapt requirements specifications of evolving systems: (NIER track). In *ICSE 2011*. 856–859.

Borges, R. V., d'Avila Garcez, A. S., and Lamb, L. C. 2010a. Integrating model verification and self-adaptation. In *ASE*. 317–320.

Borges, R. V., d'Avila Garcez, A. S., and Lamb, L. C. 2010b. Representing, learning and extracting temporal knowledge from neural networks: A case study. In *ICANN (2)*. 104–113.

Borges, R. V., d'Avila Garcez, A. S., and Lamb, L. C. 2011b. Learning and representing temporal knowledge in recurrent networks. *IEEE Transactions on Neural Networks 22,* 12, 2409–2421.

Chaki, S., Groce, A., and Strichman, O. 2004. Explaining abstract counterexamples. In *SIGSOFT FSE*. 73–82.

Chechik, M., Devereux, B., Easterbrook, S., and Gurfinkel, A. 2003. Multi-valued symbolic model-checking. *ACM Transactions on Software Engineering and Methodology 12*, 2003.

Cimatti, A., Clarke, E. M., Giunchiglia, F., and Roveri, M. 1999. Nusmv: A new symbolic model verifier. In *CAV '99*.

Cimatti, A., Pistore, M., Roveri, M., and Sebastiani, R. 2002. Improving the Encoding of LTL Model Checking into SAT. In *VMCAI'02*. 196–207.

Clarke, E., Emerson, E., and Sifakis, J. 2009. Model checking: algorithmic verification and debugging. *Comm. ACM 52,* 11, 74–84.

Clarke, E., Grumberg, O., Jha, S., Lu, Y., and Veith, H. 2003. Counterexample-guided abstraction refinement for symbolic model checking. *J. ACM 50,* 5, 752–794.

Cleland-Huang, J., Czauderna, A., Gibiec, M., and Emenecker, J. 2010. A machine learning approach for tracing regulatory codes to product specific requirements. In *ICSE 2010*. 155–164.

Damas, C., Lambeau, B., Roucoux, F., and van Lamsweerde, A. 2009. Analyzing critical process models through behavior model synthesis. In *ICSE'09*. 441–451.

d'Avila Garcez, A., Lamb, L., and Gabbay, D. 2009. *Neural-Symbolic Cognitive Reasoning*. Springer.

d'Avila Garcez, A., Zaverucha, G., and Silva, V. 1997. Applying the connectionist inductive learning and logic programming system to power systems' diagnosis. In *IEEE Intl. J. Conf. Neur. Nets. IJCNN97*. 121–126.

d'Avila Garcez, A. S., Broda, K., and Gabbay, D. M. 2002. *Neural-Symbolic Learning Systems: Foundations and Applications*. Springer.

d'Avila Garcez, A. S., Russo, A., Nuseibeh, B., and Kramer, J. 2001. An analysis-revision cycle to evolve requirements specifications. In *ASE*. 354–358.

d'Avila Garcez, A. S., Russo, A., Nuseibeh, B., and Kramer, J. 2003. Combining abductive reasoning and inductive learning to evolve requirements specifications. *IEE Proceedings - Software 150,* 1, 25–38.

Deshmukh, J., Emerson, E., and Sankaranarayanan, S. 2009. Symbolic deadlock analysis in concurrent libraries and their clients. In *ASE*. 480–491.

Dobson, S., Denazis, S., Fernández, A., Gaïti, D., Gelenbe, E., Massacci, F., P.Nixon, Saffre, F., Schmidt, N., and Zambonelli, F. 2006. A survey of autonomic communications. *ACM TAAS 1*, 223–259.

Dobson, S., Sterritt, R., Nixon, P., and Hinchey, M. 2010. Fulfilling the vision of autonomic computing. *IEEE Computer 43,* 1, 35–41.

Dumitru, H., Gibiec, M., Hariri, N., Cleland-Huang, J., Mobasher, B., Castro-Herrera, C., and Mirakhorli, M. On-demand feature recommendations derived from mining public product descriptions. In *ICSE 2011*. Honolulu, HI.

Fisher, M., Gabbay, D., and Vila, L., Eds. 2005. *Handbook of temporal reasoning in artificial intelligence*. Elsevier.

Franca, M., Zaverucha, G., and d'Avila Garcez, A. 2012. Fast relational learning using bottom clauses in neural networks. In *Proceedings of the 22nd International Conference on Inductive Logic Programming ILP2012*. Dubrovnik, Croatia, Springer LNCS.

Gabel, M. and Su, Z. 2008. Symbolic mining of temporal specifications. In *ICSE 2008*. 51–60.

Groce, A. and Kroening, D. 2005. Making the most of BMC counterexamples. *Electr. Notes Theor. Comput. Sci. 119,* 2, 67–81.

Haykin, S. 1999. *Neural Networks: A Compreensive Foundation* 2nd Ed. Prentice Hall.

Hitzler, P., Hölldobler, S., and Seda, A. 2004. Logic programs and connectionist networks. *J. Appl. Logic 2,* 3, 245–272.

Kreiker, J., Tarlecki, A., Vardi, M. Y., and Wilhelm, R. 2011. Modeling, analysis, and verification - the formal methods manifesto 2010 (Dagstuhl perspectives workshop 10482). *Dagstuhl Manifestos 1,* 1, 21–40.

Lamb, L. C., Borges, R. V., and d'Avila Garcez, A. S. 2007. A connectionist cognitive model for temporal synchronization and learning. In *AAAI-07*. AAAI Press, 827–832.

Lin, T., Horne, B., Tino, P., and Giles, C. L. 1996. Learning long-term dependencies in narx recurrent neural networks. *IEEE Trans. Neural Networks 7,* 6, 1329–1338.

Michalski, R. S. 1987. Learning strategies and automated knowledge acquisition: an overview. *Computational models of learning*, 1–19.

Muggleton, S. and Raedt, L. D. 1994. Inductive logic programming: Theory and methods. *J. Log. Program. 19/20*, 629–679.

Nuseibeh, B. 2010. Mobile privacy requirements on demand. In *Product-Focused Software Process Improvement, 11th International Conference, PROFES 2010*. 1.

Nuseibeh, B., Easterbrook, S., and Russo, A. 2000. Leveraging inconsistency in software development. *IEEE Computer 33,* 4, 24–29.

Parnas, D. L. 2010. Really rethinking 'formal methods'. *IEEE Computer 43,* 1, 28–34.

Peled, D., Vardi, M. Y., and Yannakakis, M. 2001. Black box checking. *J. of Autom. Lang. Comb. 7,* 2, 225–246.

Pnueli, A. 1977. The temporal logic of programs. In *FOCS '77*. IEEE Computer Society, 46–67.

Raedt, L. D., Frasconi, P., Kersting, K., and Muggleton, S., Eds. 2008. *Probabilistic Inductive Logic Programming - Theory and Applications*. Lecture Notes in Computer Science Series, vol. 4911. Springer.

Rumelhart, D., Hinton, G., and Williams, R. 1986. Learning representations by back-propagating errors. *Nature 323*.

Siegelmann, H., Horne, B., and Giles, C. L. 1997. Computational capabilities of recurrent NARX neural networks. *IEEE Trans. on Syst., Man, Cyb. B 27,* 2, 208–215.

Uwents, W., Monfardini, G., Blockeel, H., Gori, M., and Scarselli, F. 2011. Neural networks for relational learning: an experimental comparison. *Machine Learning 82,* 3, 315–349.

Valiant, L. G. 2003. Three problems in computer science. *Journal of ACM 50,* 1, 96–99.

Valiant, L. G. 2008. Knowledge infusion: In pursuit of robustness in artificial intelligence. In *FSTTCS*. 415–422.

Yang, B., Bryant, R. E., O'Hallaron, D. R., Biere, A., Coudert, O., Janssen, G., Ranjan, R. K., and Somenzi, F. 1998. A performance study of bdd-based model checking. In *FMCAD '98*. Springer-Verlag, London, UK, 255–289.

Zhang, D. and Tsai, J. P. 2005. *Machine Learning Applications In Software Engineering*. World Scientific.

Received October 2012; revised ??; accepted ??